

Università degli Studi di Napoli Federico II
Corso di Ingegneria del Software — a.a. 2025/26

Sistema di Prenotazione di Postazioni in Biblioteca Universitaria

Locus

Documentazione del Progetto

Gruppo 02

Leonardo Vernieri · Matteo Manco · Ernesto Manuel Girone · Alexandre Parente

Prof. Domenico Amalfitano
Corso di Laurea in Ingegneria Informatica

Indice

Sommario	
Indice	2
1. Requisiti Informali	4
1.1. Documento di Specifica Informale	4
Sistema di Prenotazione di Postazioni in Biblioteca Universitaria	4
2. Analisi e Specifica dei Requisiti.....	6
2.1. Analisi Testuale e Data Dictionary	6
2.1.1. Diagramma dei Requisiti e Catalogo RF/RNF.....	6
2.2. Diagramma dei Casi d'Uso	9
2.3. Descrizioni dei Casi d'Uso e Diagrammi di Sequenza Attore-Sistema	10
1. RegistrazioneUtente	10
2. LoginUtente	12
3. CreaSala	13
4. ModificaSala	14
5. EliminaSala.....	16
6. VisualizzaStoricoPrenotazioni.....	17
7. ConsultaFasceDisponibili	18
8. EffettuaPrenotazione.....	19
9. VisualizzaStoricoPersonale	21
10. AnnullaPrenotazione	22
11. EffettuaCheckIn	23
12. ScadenzaPrenotazione	25
13. MonitoraggioAndamentoSale	26
2.4. Matrice di Tracciabilità UC–Requisiti.....	27
2.5. Domain Model	27
2.6. Domain Model con Responsabilità GRASP	29
Introduzione dei Controller — Use Case Controller (GRASP).....	29
Introduzione dei Cataloghi — Information Expert (GRASP)	29
Postazione come Information Expert della propria disponibilità.....	29
Prenotazione come State Dependent — State Pattern (GoF).....	29
2.7. Diagrammi di Sequenza con Responsabilità.....	31
RegistrazioneUtente	31
LoginUtente	32
CreaSala	32
ConsultaFasceDisponibili	33
EffettuaCheckIn	34
EffettuaPrenotazione	35
3. Design del Sistema Software Implementato	36

3.1. Architettura Generale.....	36
3.2. Pattern Architetturale BCED.....	36
3.3. Design Pattern Adottati.....	36
3.3.1. GRASP — Assegnazione delle Responsabilità.....	36
3.3.3. Singleton Pattern (GoF) — Sessione Utente.....	37
3.3.4. Façade Pattern (GoF) — GestorePersistenza	37
3.4. Modello delle Classi — System Model	37
3.5. Snippet di Codice — Scelte Progettuali Chiave	39
3.5.1. Disponibilità Postazione — Calcolo Dinamico	39
3.5.2. Generazione FasciaOraria.....	39
3.5.3. Invio Email via Jakarta Mail	39
3.6. Diagrammi di Sequenza con architettura BCED	39
RegistrazioneUtente.....	40
LoginUtente	41
CreaSala	42
EffettuaCheckIn	44
4. Diagramma di Deploy	45
4.1. Nodi e Componenti.....	45
4.2. Dipendenze e Comunicazioni	45
5. Test Plan	46
5.1. Approccio al Testing	46
5.2. Casi di Test per Caso d'Uso.....	46
LoginUtente	46
Testing Setup	46
CreaSala	47
EffettuaPrenotazione	48
ConsultaFasceDisponibili	49
EffettuaCheckIn	50
6. Manuale Utente e Istruzioni di Installazione.....	51
6.1. Repository del Progetto.....	51
6.2. Prerequisiti	51
6.4. Build e Avvio	51
6.5. Funzionalità Principali — Guida Rapida.....	51
6.5.1. Registrazione e Login.....	51
6.5.2. Interfaccia Studente	51
6.5.3. Interfaccia Bibliotecario.....	51
6.5.4. Notifiche	52

1. Requisiti Informali

In questa sezione viene riportato il documento di specifica informale dei requisiti così come fornito dal committente. Il testo è stato oggetto di analisi testuale per l'estrazione delle candidate class, dei requisiti funzionali e dei casi d'uso.

1.1. Documento di Specifica Informale

Sistema di Prenotazione di Postazioni in Biblioteca Universitaria

Si desidera sviluppare un sistema software per la gestione della prenotazione di postazioni di studio all'interno di una biblioteca universitaria. Il sistema dovrà consentire agli studenti di prenotare una postazione in specifiche fasce orarie, visualizzare le proprie prenotazioni e accedere ai servizi della biblioteca in modo organizzato. Il sistema coinvolge due tipologie di utenti: studenti e bibliotecari, ciascuna con funzionalità e permessi specifici.

Descrizione del Sistema — Gestione utenti: Il sistema consente la registrazione di utenti tramite autenticazione con credenziali personali. Al momento della registrazione, ciascun utente deve specificare il proprio ruolo, scegliendo tra studente o bibliotecario, oltre a fornire nome, cognome, email istituzionale e numero di matricola o codice identificativo interno.

Gestione delle sale: Ogni bibliotecario ha la possibilità di creare e gestire una o più sale studio. Per ciascuna sala, mediante apposita interfaccia grafica, è possibile specificare un nome, una descrizione, il numero totale di postazioni disponibili e gli orari di apertura giornalieri. Ogni sala può essere suddivisa opzionalmente in aree differenti, ad esempio area silenziosa, area consultazione o area lavoro di gruppo. Il bibliotecario può inoltre visualizzare in ogni momento l'elenco delle prenotazioni effettuate per ciascuna sala e monitorarne lo stato.

Prenotazioni da parte degli studenti: Ogni studente autenticato dispone di una propria interfaccia nella quale può consultare le sale disponibili e verificare, per ciascun giorno, le fasce orarie prenotabili. Una prenotazione viene effettuata selezionando una sala, una data, una fascia oraria e, se prevista, una specifica area o una specifica postazione. Il sistema deve impedire che una stessa postazione venga assegnata contemporaneamente a più studenti nella medesima fascia oraria. Una volta completata la prenotazione, essa entra nello stato 'attiva'.

Gestione delle prenotazioni e check-in: Lo studente può visualizzare all'interno del proprio profilo personale l'elenco delle prenotazioni effettuate, con le relative informazioni di sala, data, orario e stato. Deve inoltre essere possibile annullare una prenotazione entro un certo limite temporale precedente all'inizio della fascia prenotata. In caso di annullamento, la postazione torna automaticamente disponibile per altri utenti. Nel giorno della prenotazione, lo studente può effettuare il check-in tramite apposita interfaccia grafica, selezionando la prenotazione attiva e confermando la propria presenza. Il check-in deve essere effettuato entro un intervallo di tempo limitato rispetto all'orario di inizio della prenotazione. Se il check-in non viene effettuato entro tale intervallo, la prenotazione passa automaticamente nello stato 'scaduta' e la postazione viene resa nuovamente disponibile.

Monitoraggio (Bibliotecario): Il bibliotecario può consultare in tempo reale la situazione delle sale, visualizzando il numero di postazioni occupate, il numero di postazioni libere e l'elenco delle prenotazioni attive per la giornata corrente. Deve inoltre essere possibile accedere a uno storico delle prenotazioni per ciascuna sala e per ciascuno studente. I bibliotecari possono monitorare l'andamento complessivo del

servizio, visualizzando il tasso di occupazione delle sale, il numero di prenotazioni giornaliere e il numero di prenotazioni non confermate tramite check-in.

Profilo studente: Ogni studente ha accesso a un profilo personale che consente di consultare le prenotazioni future, quelle passate, quelle annullate ed eventualmente il numero totale di accessi effettuati alla biblioteca.

Notifiche e requisiti non funzionali: È previsto un sistema di notifiche che avvisi gli studenti della conferma della prenotazione, di eventuali annullamenti e dell'approssimarsi dell'orario prenotato. Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva. Il sistema dovrà garantire la protezione dei dati personali e una corretta gestione dei permessi e delle visibilità tra studenti e bibliotecari.

2. Analisi e Specifica dei Requisiti

In questa sezione vengono riportati gli artefatti prodotti durante la fase di analisi e specifica dei requisiti. La sequenza seguita è: analisi testuale e catalogazione dei requisiti → diagramma dei casi d'uso con descrizioni e diagrammi di sequenza attore-sistema → domain model → domain model con responsabilità → diagrammi di sequenza con responsabilità → matrice di tracciabilità UC–Requisiti.

2.1. Analisi Testuale e Data Dictionary

L'analisi testuale del documento di specifica informale costituisce il punto di partenza dell'intera fase di analisi e specifica. Il testo è stato esaminato sistematicamente in Visual Paradigm, evidenziando mediante sottolineatura e codice colore i termini rilevanti. Il processo ha prodotto tre categorie di artefatti:

- **Candidate class** — i sostantivi e sintagmi nominali significativi, ottenuti eliminando duplicati e attributi impropriamente nominati come entità. Dal raffinamento sono emerse le seguenti entità di dominio: Studente, Bibliotecario, SalaStudio, Area, Postazione, Prenotazione.
- **Casi d'uso** — le interazioni tra attori e sistema individuate a partire dai verbi e dalle frasi d'azione del testo. Dall'analisi sono stati identificati anche gli attori coinvolti: Studente, Bibliotecario, Utente e TimerDiSistema come attori primari, e Sistema di notifiche come attore secondario. I casi d'uso sono successivamente formalizzati nel diagramma (§ 2.2) e nelle relative descrizioni tabellari (§ 2.3).
- **Requisiti di sistema** — le funzionalità e i vincoli espressi nel testo, classificati in requisiti funzionali (RF_01–RF_24) e non funzionali (RNF_01–RNF_02), catalogati nel diagramma dei requisiti (§ 2.1.1).

2.1.1. Diagramma dei Requisiti e Catalogo RF/RNF

A partire dall'analisi testuale del documento di specifica informale, sono stati identificati e formalizzati 24 requisiti funzionali (RF_01–RF_24) e 2 requisiti non funzionali (RNF_01–RNF_02), modellati in Visual Paradigm. La tabella seguente riporta il catalogo completo con identificativo, tipo e descrizione.

ID	Nome	Tipo	Descrizione
RF_01	RegistrazioneUtente	RF	Il sistema deve permettere ad un nuovo utente di registrarsi nel sistema permettendogli di scegliere tra il ruolo di Studente e Bibliotecario e fornendo le proprie credenziali: nome, cognome, e-mail istituzionale, password e numero di matricola (Studente) o codice identificativo interno (Bibliotecario).
RF_02	LoginUtente	RF	Il sistema deve permettere ad un utente già registrato di accedere al sistema inserendo e-mail istituzionale e password; se le credenziali non corrispondono ad alcun account registrato, il sistema deve notificare l'errore.
RF_03	CreaSala	RF	Il sistema deve consentire ad un bibliotecario autenticato di aggiungere una sala inserendo nome, descrizione, numero totale di postazioni, orari di apertura giornalieri e, opzionalmente, una suddivisione interna in aree differenti.
RF_04	ModificaSala	RF	Il sistema deve permettere a un bibliotecario autenticato di modificare i dati di una sala esistente, inclusi nome,

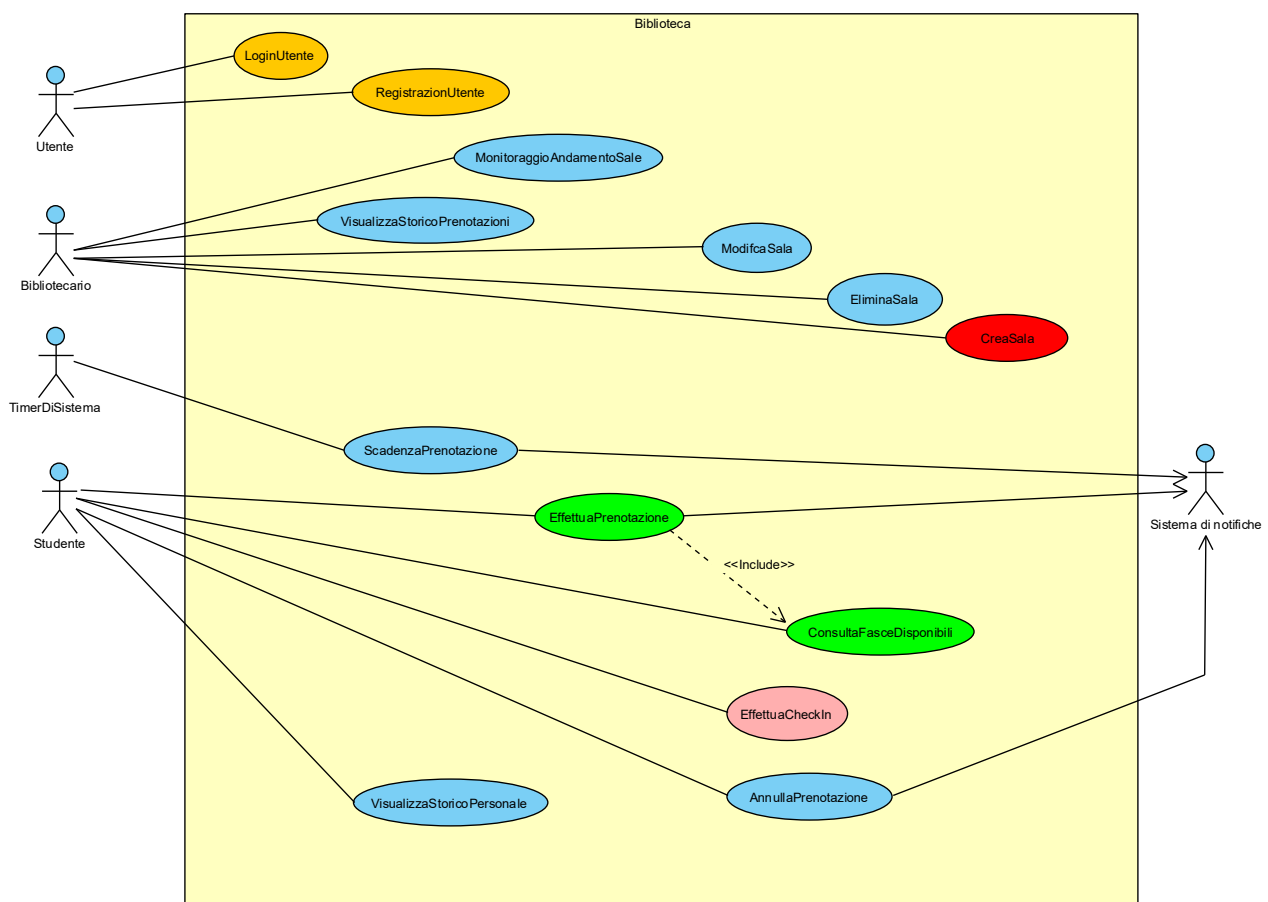
			descrizione, numero di postazioni, orari di apertura e suddivisione in aree interne.
RF_05	EliminaSala	RF	Il sistema deve permettere ad un bibliotecario autenticato di eliminare una sala esistente; se alla sala sono associate prenotazioni future, il sistema deve impedire l'eliminazione e notificare il bibliotecario delle prenotazioni pendenti.
RF_06	VisualizzaStoricoPrenotazioni	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare l'elenco delle prenotazioni effettuate selezionando una sala; per ciascuna prenotazione deve mostrare lo stato, la data, la postazione e lo studente.
RF_07	ConsultaFasceDisponibili	RF	Il sistema deve permettere ad uno studente autenticato di selezionare una sala, un giorno e un'area, e visualizzare l'elenco delle fasce orarie disponibili.
RF_08	RegistraPrenotazione ¹	RF	Il sistema deve permettere ad uno studente autenticato di registrare una prenotazione inserendo le informazioni relative alla sala, data, fascia oraria e, opzionalmente, una specifica area o postazione.
RF_09	VisualizzaPrenotazioniPersonalì	RF	Il sistema deve permettere ad uno studente autenticato di visionare l'elenco delle proprie prenotazioni passate, annullate e future, con informazioni di sala, data, orario e stato all'interno del profilo personale.
RF_10	AnnullaPrenotazione	RF	Il sistema deve permettere ad uno studente autenticato di annullare una propria prenotazione attiva, a condizione che l'annullamento avvenga entro un limite di tempo definito prima dell'inizio della fascia oraria prenotata; in caso di annullamento la postazione torna automaticamente disponibile.
RF_11	EffettuaCheckIn	RF	Il sistema deve permettere ad uno studente autenticato di effettuare il check-in per una propria prenotazione tramite apposita interfaccia, entro 2 ore dall'inizio della prenotazione.
RF_12	ScadenzaPrenotazione	RF	Il sistema deve gestire automaticamente il caso di prenotazione scaduta: se il check-in non avviene entro 2 ore dalla prenotazione, il sistema aggiorna lo stato in 'scaduta' e rende la postazione nuovamente disponibile.
RF_13	VisualizzaPostazioniOccupate	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare il numero di postazioni occupate in tempo reale in un'interfaccia dedicata.

RF_14	VisualizzaNumeroPostazioniLibere	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare il numero di postazioni libere di tutte le sale in tempo reale.
RF_15	VisualizzaElencoPrenotazioniAttive	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare l'elenco delle prenotazioni attive di tutte le sale in tempo reale.
RF_16	VisualizzaStoricoPrenotazioni	RF	Il sistema deve permettere ad un bibliotecario autenticato di accedere all'elenco di tutte le prenotazioni effettuate, specificando una sala o uno studente.
RF_17	VisualizzaNumeroTotaleAccessi	RF	Il sistema deve permettere ad uno studente autenticato di visualizzare il numero totale di accessi effettuati alla biblioteca nel proprio profilo personale.
RF_18	VisualizzaTassoOccupazione	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare il tasso di occupazione delle sale nell'interfaccia di monitoraggio.
RF_19	VisualizzaNumeroPrenotazioniGiornaliere	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare il numero di prenotazioni giornaliere delle sale nell'interfaccia di monitoraggio.
RF_20	VisualizzaNumeroPrenotazioniNonConfermate	RF	Il sistema deve permettere ad un bibliotecario autenticato di visualizzare il numero di prenotazioni non confermate tramite check-in nell'interfaccia di monitoraggio.
RF_21	InviaNotificaApprossimarsiPrenotazione	RF	Il sistema deve notificare lo studente il giorno prima dell'approssimarsi della propria prenotazione.
RF_22	InviaNotificaAnnullamentoPrenotazione	RF	Il sistema deve notificare lo studente di un annullamento della propria prenotazione specificando la causa.
RF_23	InviaNotificaConfermaPrenotazione	RF	Il sistema deve notificare lo studente del completamento di una propria prenotazione.
RF_24	VerificaPrenotazioneOccupata	RF	Il sistema deve impedire che una stessa postazione in una medesima fascia oraria venga assegnata contemporaneamente a più studenti.
RNF_01	Accessibilità Web/Mobile	RNF	Il sistema dovrà essere accessibile via web sia da desktop che da dispositivi mobili, con un'interfaccia grafica intuitiva e responsiva.
RNF_02	ProtezioneDatiGDPR	RNF	Il sistema deve garantire la protezione dei dati personali degli utenti in conformità con la normativa GDPR, assicurando che ogni utente possa accedere

esclusivamente alle funzionalità e ai dati previsti dal proprio ruolo (studente o bibliotecario).

2.2. Diagramma dei Casi d'Uso

Il diagramma dei casi d'uso mostra l'insieme completo delle funzionalità offerte dal sistema, organizzate per attore. Gli attori primari sono **Studente**, **Bibliotecario**, **Utente** (visione dello studente o bibliotecario non ancora autenticati) e **TimerDiSistema**; l'attore secondario è **Sistema di notifiche**. Tra i tredici casi d'uso identificati, sei sono stati selezionati per l'implementazione: **EffettuaPrenotazione**, **ConsultaFasceDisponibili**, **LoginUtente**, **RegistrazioneUtente**, **CreaSala** ed **EffettuaCheckIn**. Nella sezione successiva (§ 2.3) vengono riportate le descrizioni tabellari e i diagrammi di sequenza attore-sistema per questi sei casi d'uso.



I principali raggruppamenti funzionali sono:

- Funzionalità comuni (Utente): **RegistrazioneUtente**, **LoginUtente**.
- Gestione Sale (Bibliotecario): **CreaSala**, **ModificaSala**, **EliminaSala**, **MonitoraggioAndamentoSale**, **VisualizzaStoricoPrenotazioni**.
- Prenotazioni (Studente): **ConsultaFasceDisponibili**, **EffettuaPrenotazione**, **AnnullaPrenotazione**, **EffettuaCheckIn**, **VisualizzaStoricoPersonale**.
- Automazione (TimerDiSistema): **ScadenzaPrenotazione** (con notifiche via Sistema di notifiche).

2.3. Descrizioni dei Casi d'Uso e Diagrammi di Sequenza Attore-Sistema

Di seguito vengono riportate le descrizioni in forma tabellare dei casi d'uso identificati, con precondizioni, postcondizioni, scenario principale e tracciabilità ai requisiti.

1. RegistrazioneUtente

Use Case	RegistrazioneUtente
ID	UC_01
Attori Primari	Utente (non ancora registrato)
Precondizioni	L'utente possiede codice identificativo o matricola e non ha ancora un account nel sistema.
Postcondizioni	Esiste un nuovo account con le credenziali fornite; l'utente può effettuare il login.
Scenario Base	1. Utente seleziona l'opzione di registrazione. 2. loop 2.1. SYSTEM mostra il form con i campi: nome, cognome, email, password, ruolo. 2.2. Utente seleziona un ruolo. 2.3. if il ruolo selezionato è "Studente" 2.3.1. SYSTEM mostra il campo aggiuntivo: matricola. 2.4. else if il ruolo selezionato è "Bibliotecario" 2.4.1. SYSTEM mostra il campo aggiuntivo: codice identificativo interno. 2.5. Utente inserisce le informazioni. 2.6. SYSTEM controlla i dati inseriti. 2.7. if dati inseriti non validi o mancanti 2.7.1. SYSTEM notifica l'utente che i dati inseriti sono errati o mancanti. 3. SYSTEM memorizza il nuovo account con il ruolo scelto. 4. SYSTEM notifica l'utente che la registrazione è avvenuta con successo. 5. SYSTEM reindirizza a LoginUtente.
Requisiti coperti	RF_01
Note	Autore: Ernesto Manuel Girone

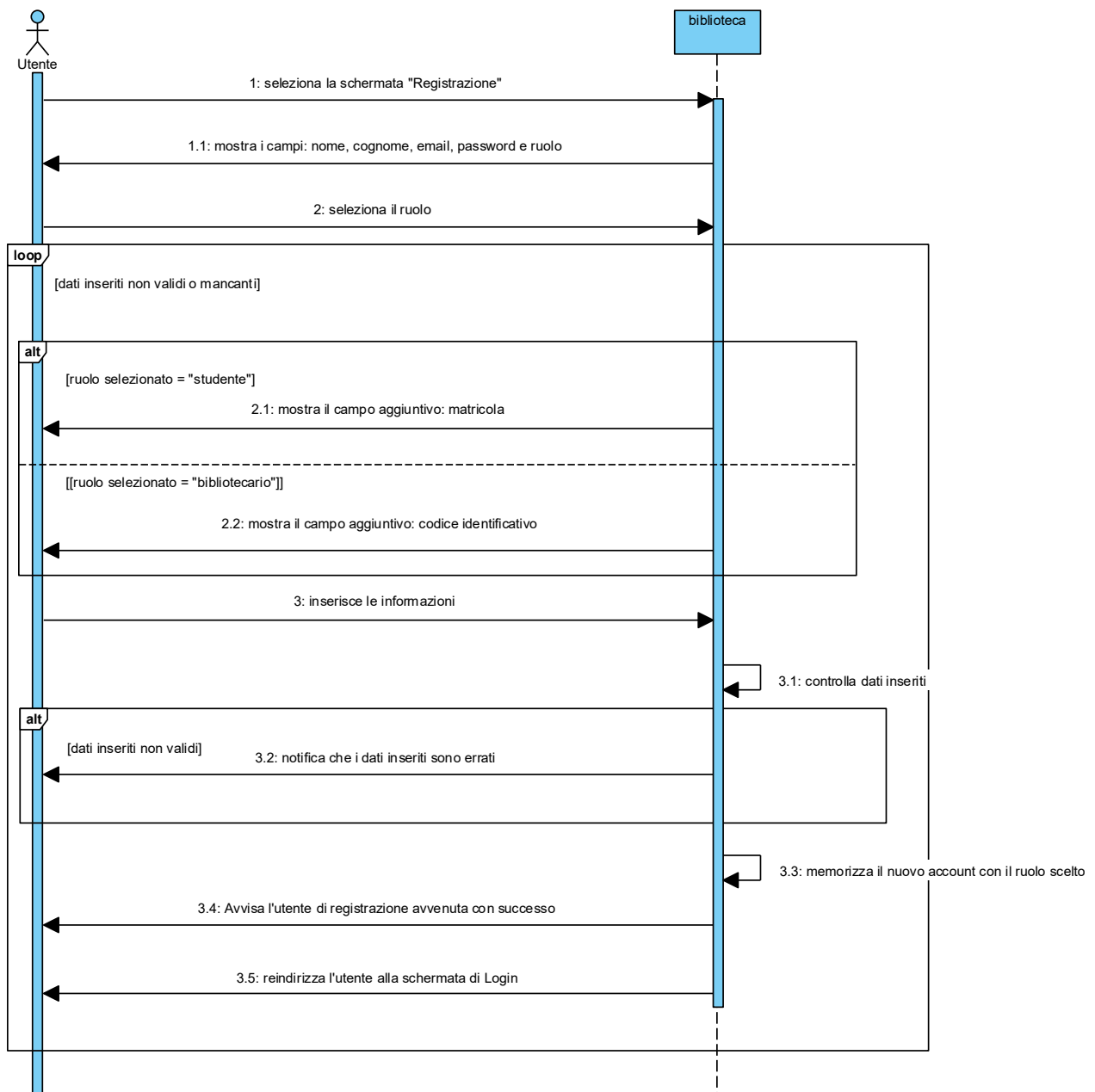


Figura 1 Sequence diagram Attore-Sistema -- Registrazione Utente

2. LoginUtente

Use Case	LoginUtente
ID	UC_02
Attori Primari	Utente (registrato)
Precondizioni	L'utente è già registrato nel sistema.
Postcondizioni	L'utente è autenticato e può eseguire le azioni specifiche per il proprio ruolo.
Scenario Base	1. Utente seleziona l'opzione di login. 2. loop 2.1. SYSTEM mostra il form con i campi: email e password. 2.2. Utente inserisce le credenziali e conferma. 2.3. SYSTEM controlla le credenziali inserite. until credenziali valide oppure tentativi ≥ 3 3. if tentativi massimi esauriti 3.1. SYSTEM avvisa l'utente: tentativi massimi raggiunti — credenziali errate. 3.2. exit Scenario Base. - end if 4. SYSTEM reindirizza l'utente alla pagina iniziale associata al proprio ruolo
Requisiti coperti	RF_02
Note	Autore: Ernesto Manuel Girone

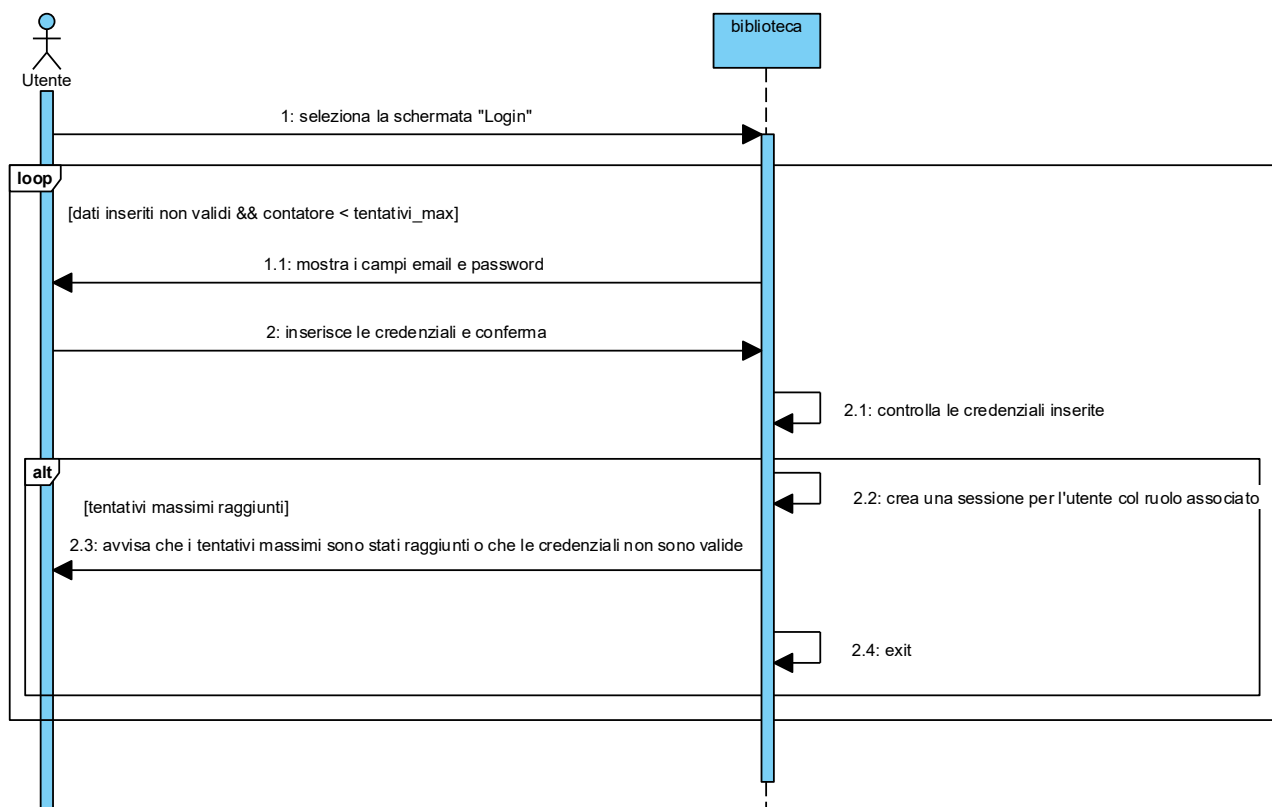


Figura 2 Sequence Diagram Attore-Sistema — LoginUtente

3. CreaSala

Use Case	CreaSala
ID	UC_03
Attori Primari	Bibliotecario
Precondizioni	L'utente è autenticato col ruolo di bibliotecario.
Postcondizioni	Una sala viene creata, memorizzata e associata al bibliotecario.
Scenario Base	- Bibliotecario seleziona l'opzione per la creazione della sala. - SYSTEM mostra il form con i campi: nome, descrizione, numero totale di postazioni, orari di apertura giornalieri, presenza di aree. - Bibliotecario inserisce le informazioni. - if presenza di aree = vero - Bibliotecario inserisce numero e tipologia delle aree. end if - Bibliotecario conferma. - if dati corretti - SYSTEM memorizza la sala e mostra conferma. else - SYSTEM mostra un avviso per i dati non corretti. end if
Requisiti coperti	RF_03
Note	Autore: Leonardo Vernieri

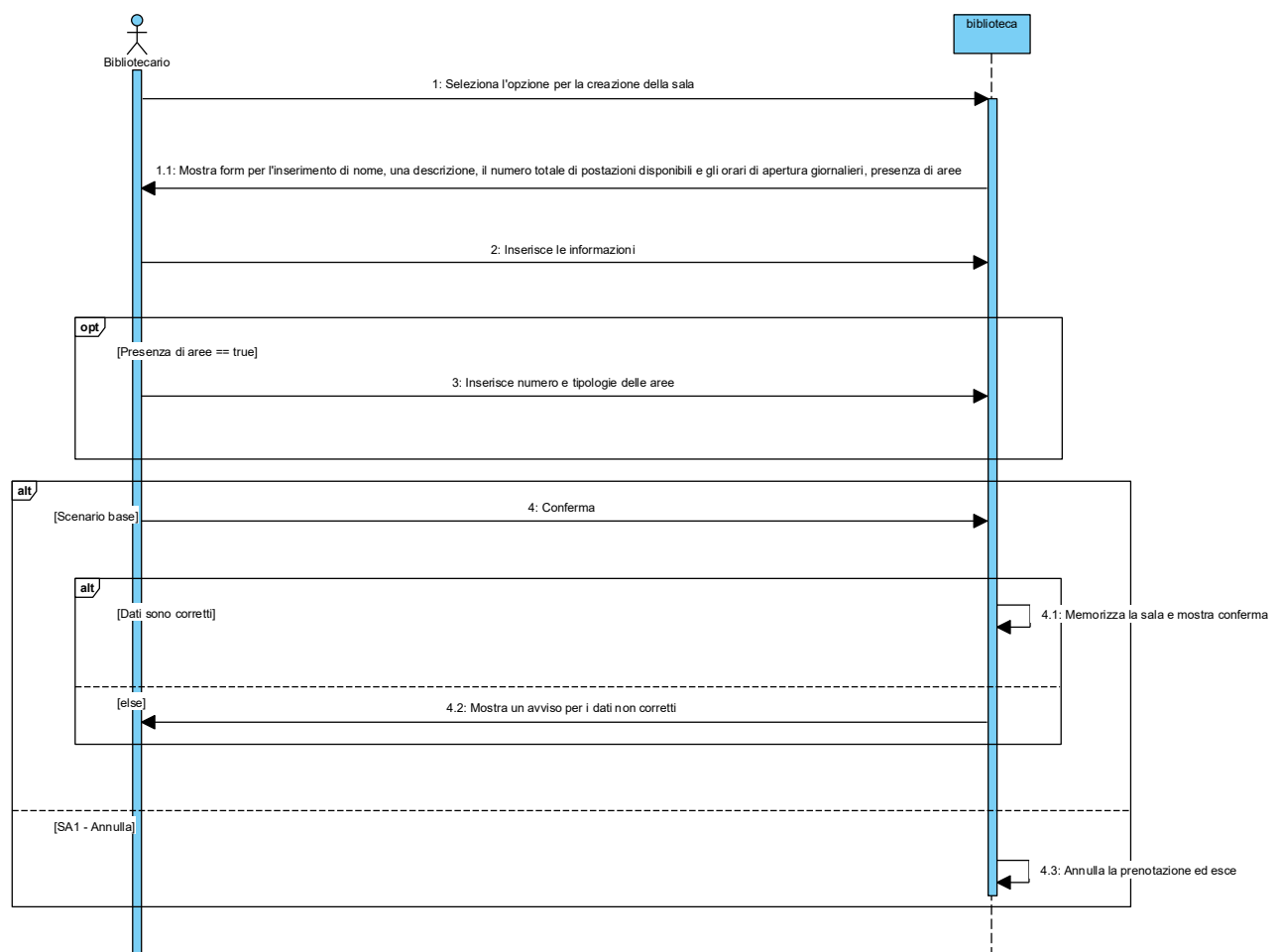


Figura 3 Sequence Diagram attore-sistema — CreaSala

4. ModificaSala

Use Case	ModificaSala
ID	UC_04
Attori Primari	Bibliotecario
Precondizioni	L'utente è autenticato col ruolo di bibliotecario.
Postcondizioni	I campi della sala selezionata vengono aggiornati.
Scenario Base	1. Bibliotecario seleziona l'opzione per la modifica della sala. 2. SYSTEM recupera tutte le sale disponibili. 3. if non ci sono sale 3.1. SYSTEM segnala la mancanza di sale nel sistema. 3.2. exit Scenario Base. - end if 4. SYSTEM mostra l'elenco delle sale. 5. Bibliotecario seleziona la sala da modificare. 6. SYSTEM recupera e mostra i dati attuali della sala. 7. loop 7.1. Bibliotecario modifica i campi desiderati. 7.2. SYSTEM verifica che le informazioni siano corrette. 7.3. if informazioni non corrette 7.3.1. SYSTEM notifica l'informazione non corretta. - end if - until informazioni corrette 8. SYSTEM chiede di confermare la modifica. 9. Bibliotecario inserisce la risposta. 10. if risposta = sì 10.1. SYSTEM aggiorna la sala. 10.2. SYSTEM mostra la vista della sala con le informazioni aggiornate. 11. else 11.1. SYSTEM annulla le modifiche. - end if
Requisiti coperti	RF_04

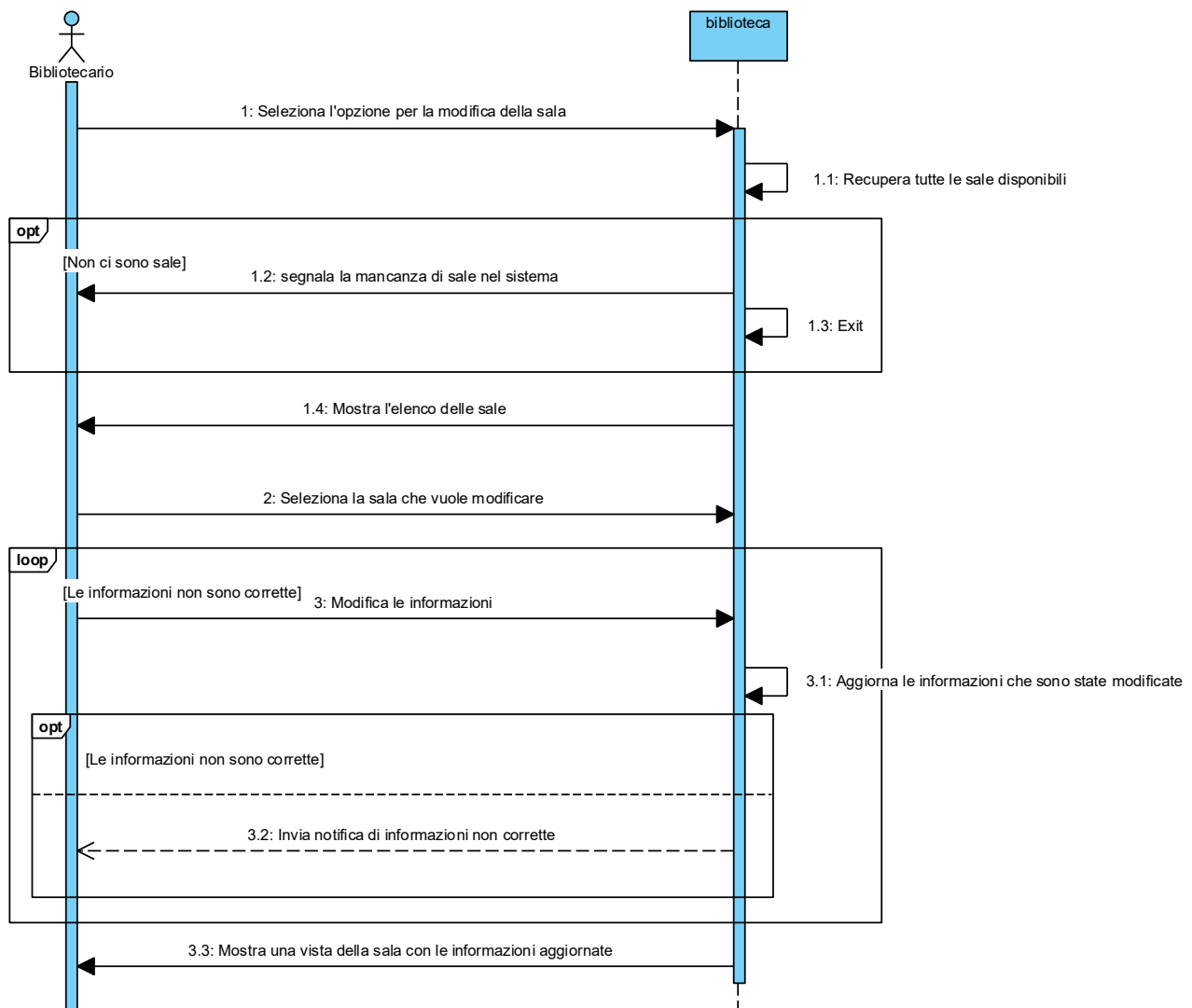


Figura 4 Sequence Diagram attore-sistema — ModificaSala

5. EliminaSala

Use Case	EliminaSala
ID	UC_05
Attori Primari	Bibliotecario
Precondizioni	L'utente è autenticato col ruolo di bibliotecario.
Postcondizioni	La sala selezionata viene eliminata dal sistema.
Scenario Base	- Bibliotecario seleziona l'opzione per l'eliminazione della sala. - SYSTEM recupera tutte le sale disponibili. - if non ci sono sale 3.1. SYSTEM segnala la mancanza di sale nel sistema. 3.2. exit Scenario Base. - end if - SYSTEM mostra l'elenco delle sale. - Bibliotecario seleziona la sala da eliminare. - SYSTEM chiede conferma dell'eliminazione. - Bibliotecario seleziona la risposta. - if risposta = sì 8.1. SYSTEM elimina la sala. - end if - SYSTEM mostra l'elenco delle sale aggiornato.
Requisiti coperti	RF_05

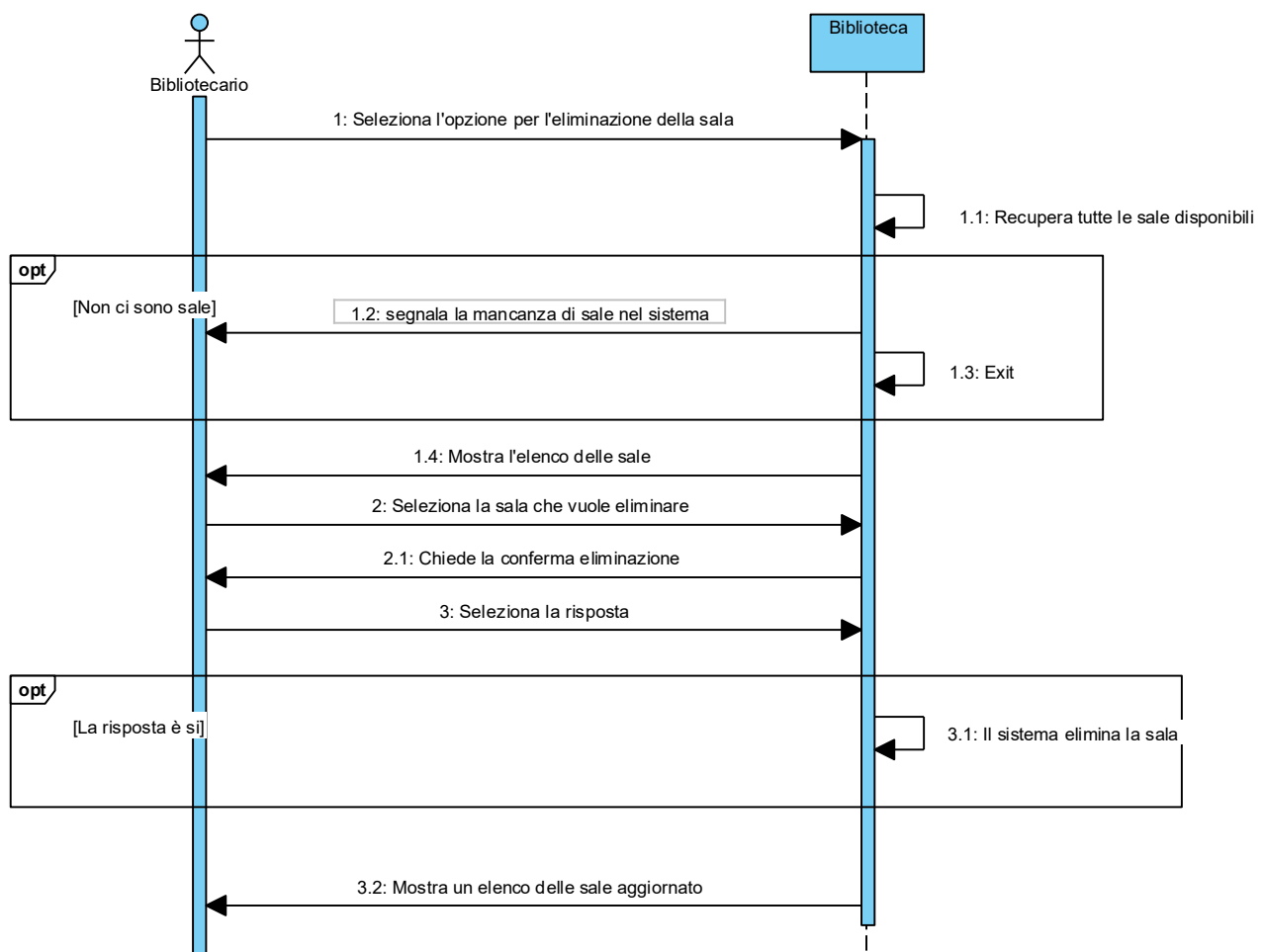


Figura 5 Sequence Diagram attore-sistema — EliminaSala

6. VisualizzaStoricoPrenotazioni

Use Case	VisualizzaStoricoPrenotazioni
ID	UC_06
Attori Primari	Bibliotecario
Precondizioni	L'utente è autenticato col ruolo di bibliotecario.
Postcondizioni	Nessun dato modificato.
Scenario Base	1. Bibliotecario accede alla schermata "Storico Prenotazioni". 2. Bibliotecario seleziona una sala o uno studente e una data. 3. SYSTEM recupera le prenotazioni effettuate relative al filtro selezionato. 4. if non trovata alcuna prenotazione 4.1. SYSTEM mostra un messaggio di assenza di prenotazioni. 5. else 5.1. SYSTEM mostra l'elenco delle prenotazioni trovate con le relative informazioni: fascia oraria, data, stato e, se prevista, area o postazione specifica. end if
Requisiti coperti	RF_06

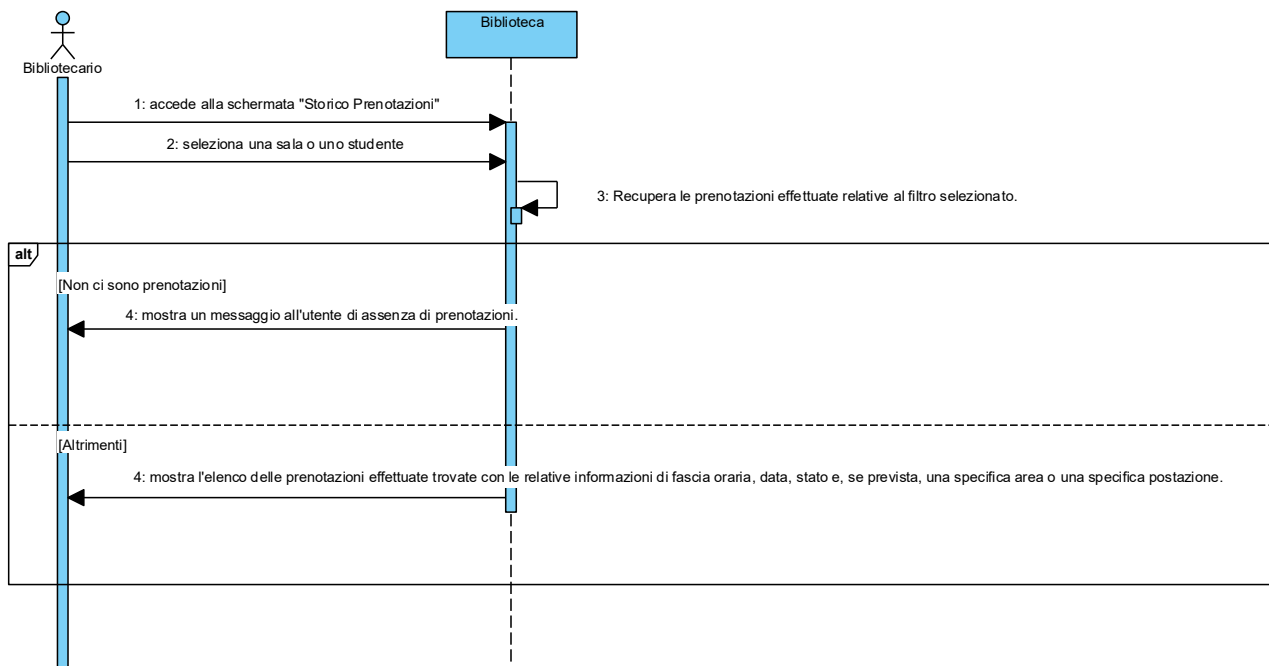


Figura 6 Sequence Diagram attore-sistema — VisualizzaStoricoPrenotazioni

7. ConsultaFasceDisponibili

Use Case	ConsultaFasceDisponibili
ID	UC_07
Attori Primari	Studente
Precondizioni	L'utente è autenticato col ruolo di studente.
Postcondizioni	Nessun dato modificato.
Scenario Base	1. Studente seleziona "Consulta Fasce Orarie prenotabili" dal menu. 2. Studente seleziona una sala, un giorno e un'area. 3. SYSTEM recupera il numero di postazioni disponibili per fascia oraria per la sala, il giorno e l'area selezionati. 4. if non trovata alcuna fascia oraria disponibile 4.1. SYSTEM mostra un messaggio di assenza di fasce orarie disponibili. 5. else 5.1. SYSTEM mostra l'elenco delle fasce orarie trovate con il numero di postazioni disponibili rispetto al totale. end if
Requisiti coperti	RF_07, RF_24
Note	Autore: Matteo Manco. Use case incluso da EffettuaPrenotazione.

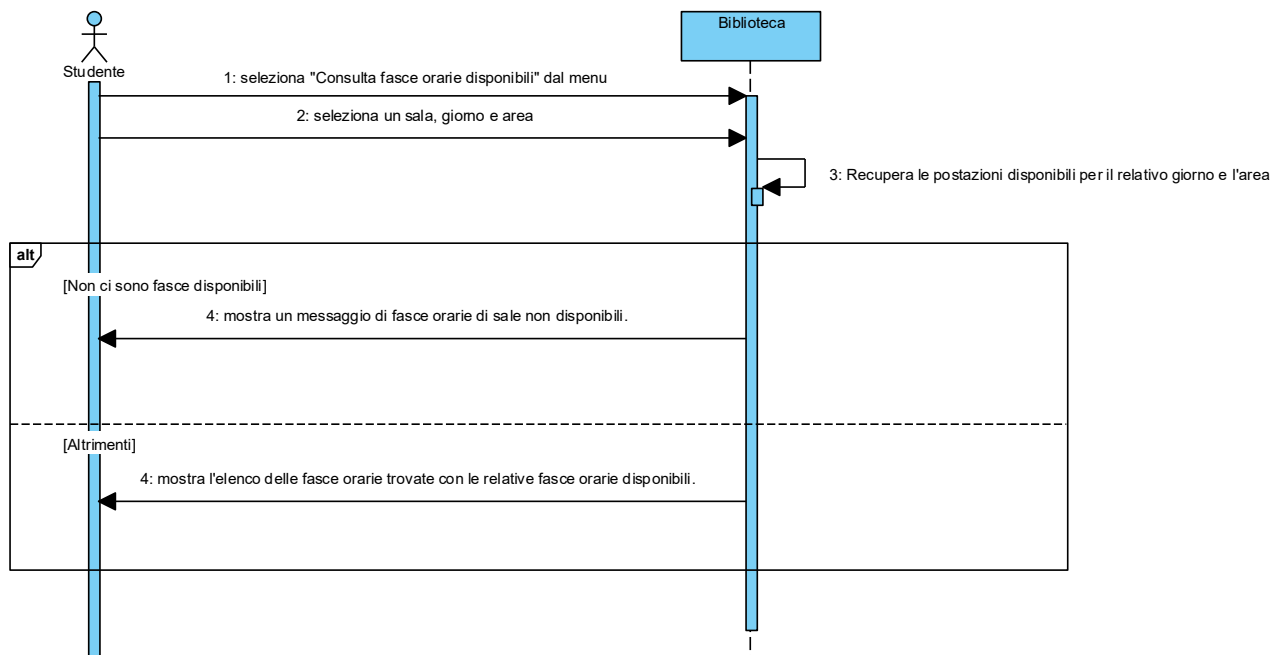


Figura 7 Sequence Diagram attore-sistema — ConsultaFasceDisponibili

8. EffettuaPrenotazione

Use Case	EffettuaPrenotazione
ID	UC_08
Attori Primari	Studente, Sistema di notifiche
Precondizioni	L'utente è autenticato nel ruolo di studente.
Postcondizioni	Una o più prenotazioni vengono create, memorizzate e associate all'utente.
Scenario Base	1. Studente seleziona "Effettua Prenotazione". 2. Include(ConsultaFasceDisponibili). 3. Studente seleziona una o più fasce orarie da quelle mostrate. 4. SYSTEM richiede conferma prenotazione. 5. if Studente conferma 5.1. SYSTEM registra la prenotazione. 5.2. SYSTEM mostra conferma della registrazione nell'interfaccia. 5.3. SYSTEM invoca Sistema di notifiche per inviare notifica allo Studente. 6. else if Studente non conferma 6.1. exit Scenario Base. end if
Scenario Alternativo	SA1 — Fasce Orarie Contigue Al punto 3 dello Scenario Base, lo Studente seleziona più fasce orarie contigue. 3.1. SYSTEM ricerca una postazione disponibile per tutte le fasce orarie selezionate. 4. if postazione unica non trovata 4.1. SYSTEM ricerca postazioni distinte per soddisfare le fasce su prenotazioni separate. 4.2. SYSTEM richiede conferma di procedere con due prenotazioni su postazioni diverse. 5. else 5.1. SYSTEM richiede conferma prenotazione. end if 6. if Studente conferma 6.1. SYSTEM registra la/le prenotazione/i. 6.2. SYSTEM mostra conferma della registrazione nell'interfaccia. 6.3. SYSTEM invoca Sistema di notifiche per inviare notifica allo Studente. 7. else if Studente non conferma 7.1. exit Scenario Alternativo. end if
Requisiti coperti	RF_08, RF_23, RF_24
Note	Autore: Matteo Manco. Estende ConsultaFasceDisponibili via <<include>>.

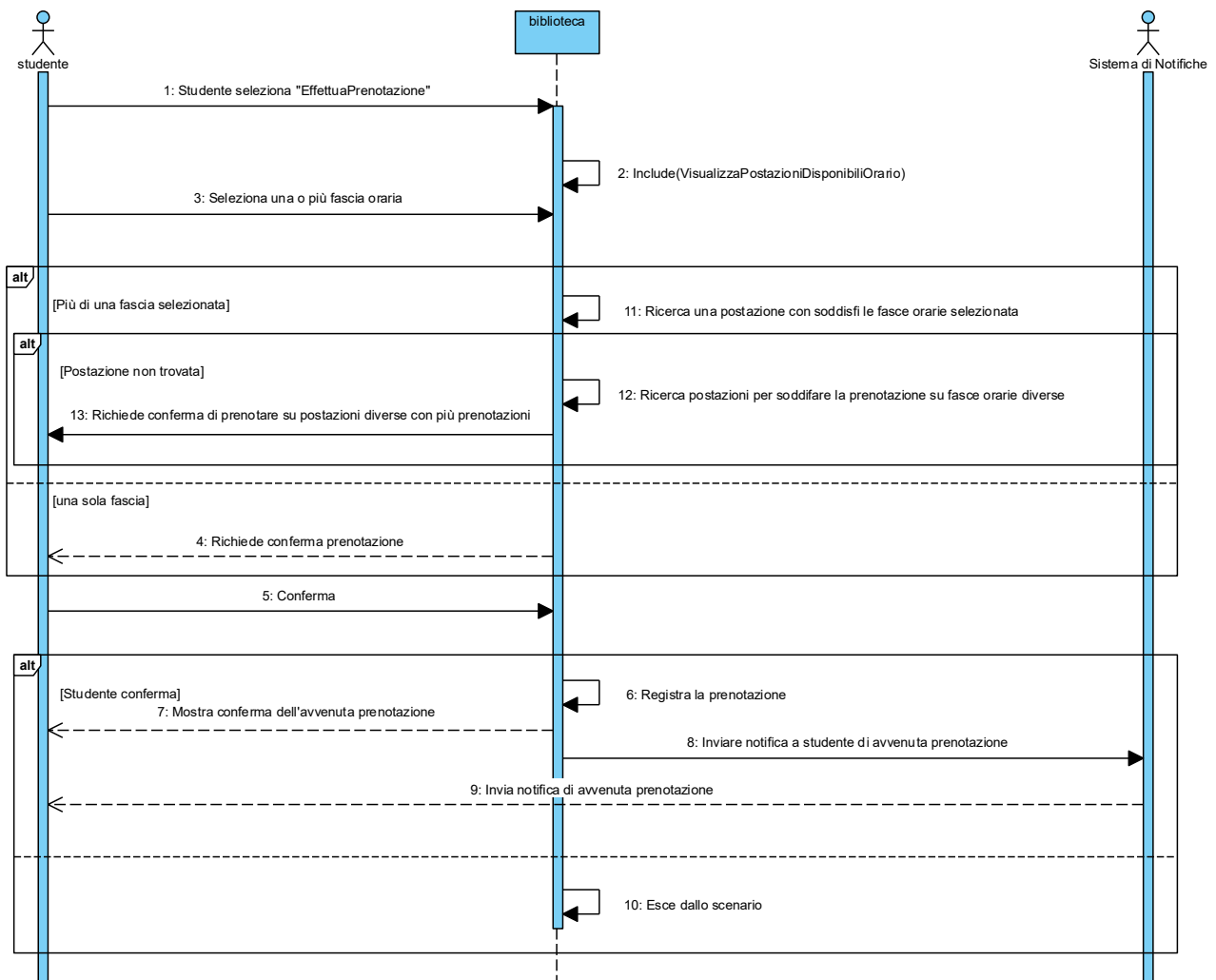


Figura 8 Sequence Diagram attore-sistema — EffettuaPrenotazione

9. VisualizzaStoricoPersonale

Use Case	VisualizzaStoricoPersonale
ID	UC_09
Attori Primari	Studente
Precondizioni	L'utente è autenticato col ruolo di studente.
Postcondizioni	Nessun dato modificato.
Scenario Base	1. Studente accede alla schermata del Profilo Personale. 2. SYSTEM recupera le prenotazioni associate allo Studente. 3. if non trovata alcuna prenotazione associata 3.1. SYSTEM mostra il numero totale di accessi effettuati (0). 3.2. SYSTEM mostra un messaggio di assenza di prenotazioni. 4. else 4.1. SYSTEM calcola il numero di accessi effettuati in biblioteca dallo Studente. 4.2. SYSTEM mostra l'elenco delle prenotazioni trovate con le relative informazioni: sala, data, orario e stato. 4.3. SYSTEM mostra il numero totale di ingressi effettuati in biblioteca. end if
Requisiti coperti	RF_09

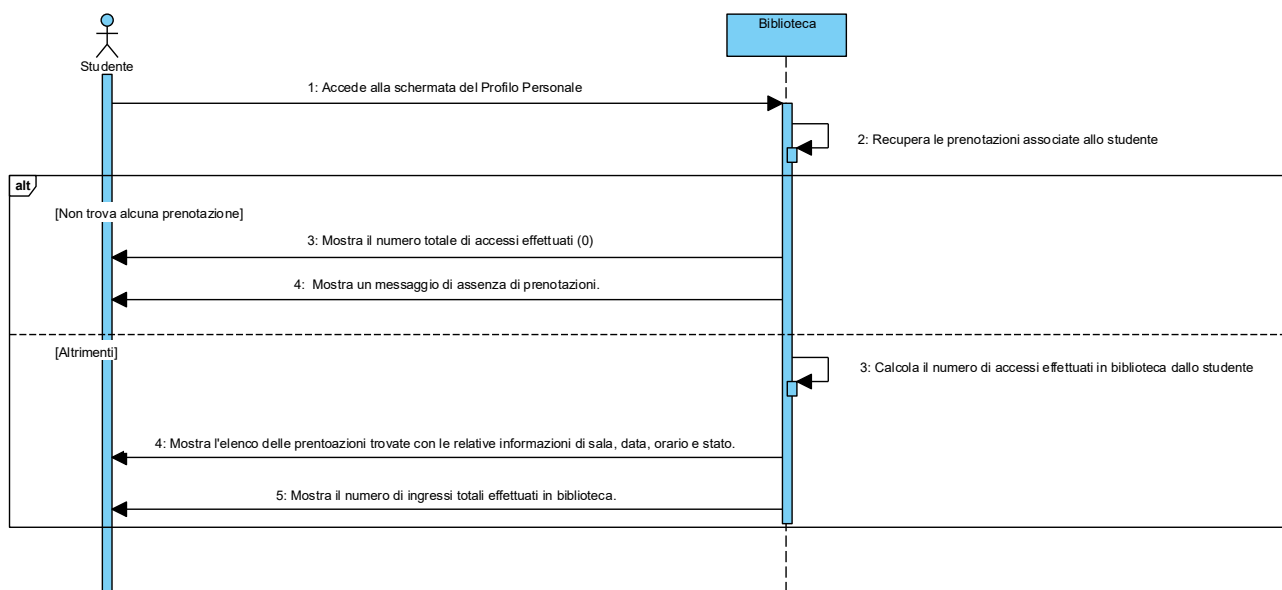


Figura 9 Sequence Diagram attore-sistema — VisualizzaPrenotazioniPersonalì

10. AnnullaPrenotazione

Use Case	AnnullaPrenotazione
ID	UC_10
Attori Primari	Studente, Sistema di notifiche
Precondizioni	L'utente è autenticato col ruolo di studente e ha almeno una prenotazione attiva.
Postcondizioni	La prenotazione passa allo stato 'annullata'; la postazione viene resa nuovamente disponibile.
Scenario Base	1. Studente seleziona "Le mie prenotazioni". 2. SYSTEM recupera le prenotazioni attive associate allo Studente. 3. SYSTEM mostra l'elenco delle prenotazioni dello Studente. 4. Studente seleziona la prenotazione da annullare. 5. SYSTEM richiede conferma dell'annullamento. 6. Studente risponde. 7. if Studente risponde no 7.1. Operazione annullata. 7.2. exit Scenario Base. - end if 8. SYSTEM aggiorna lo stato della prenotazione in "annullata". 9. SYSTEM rende disponibile la postazione. 10. SYSTEM mostra conferma dell'annullamento nell'interfaccia allo Studente. 11. SYSTEM invoca Sistema di notifiche per inviare notifica allo Studente dell'annullamento della prenotazione.
Requisiti coperti	RF_10, RF_22

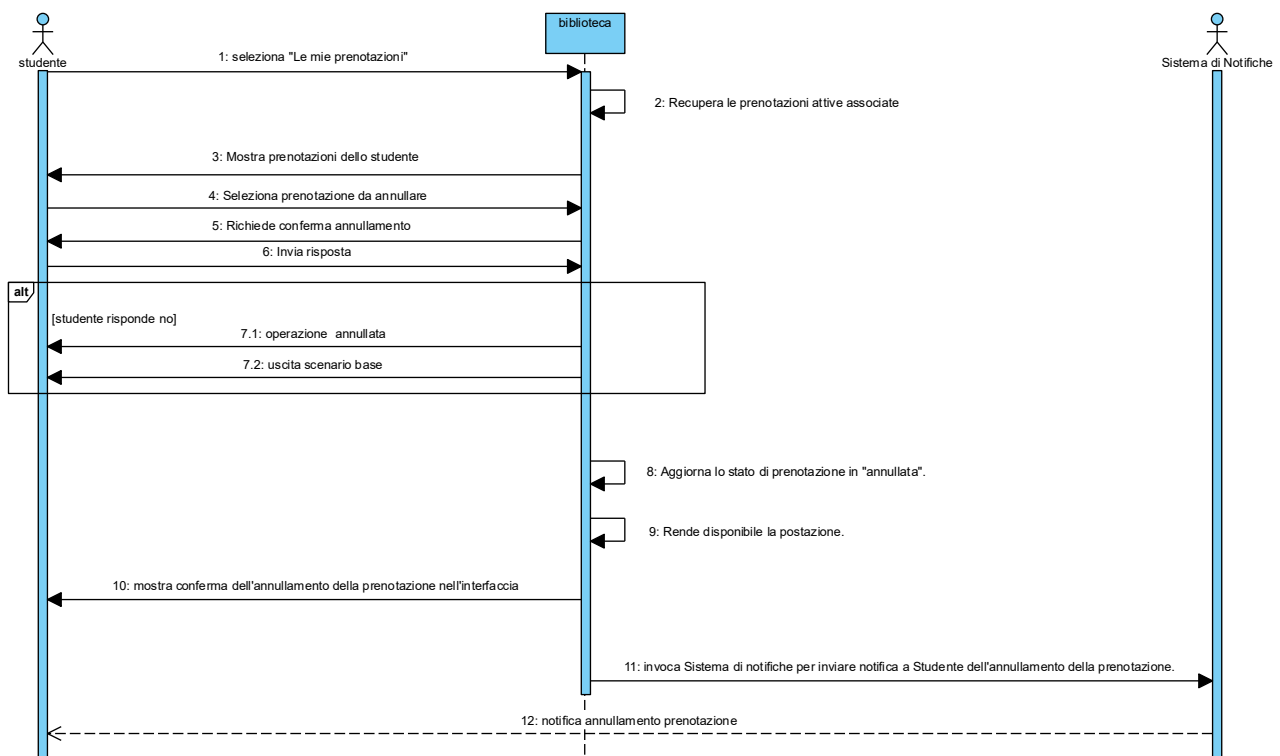


Figura 10 Sequence Diagram attore-sistema — AnnullaPrenotazione

11. EffettuaCheckIn

Use Case	EffettuaCheckIn
ID	UC_11
Attori Primari	Studente, Sistema di notifiche
Precondizioni	L'utente è autenticato col ruolo di studente.
Postcondizioni	La prenotazione passa allo stato 'confermata'.
Scenario Base	1. Studente seleziona "Effettua Check-In". 2. SYSTEM recupera le prenotazioni attive associate allo Studente. 3. SYSTEM mostra l'elenco delle prenotazioni dello Studente. 4. Studente seleziona la prenotazione su cui effettuare il check-in. 5. if prenotazione scaduta 5.1. SYSTEM mostra messaggio: impossibile effettuare il check-in per prenotazione scaduta. 5.2. exit Scenario Base. - end if 6. SYSTEM chiede conferma check-in. 7. Studente seleziona sì o no. 8. if risposta = sì 8.1. SYSTEM aggiorna la prenotazione in "confermata". 8.2. SYSTEM mostra conferma del check-in nell'interfaccia allo Studente. 8.3. SYSTEM invoca Sistema di notifiche per inviare notifica allo Studente. - end if
Requisiti coperti	RF_11
Note	Autore: Alexandre Parente

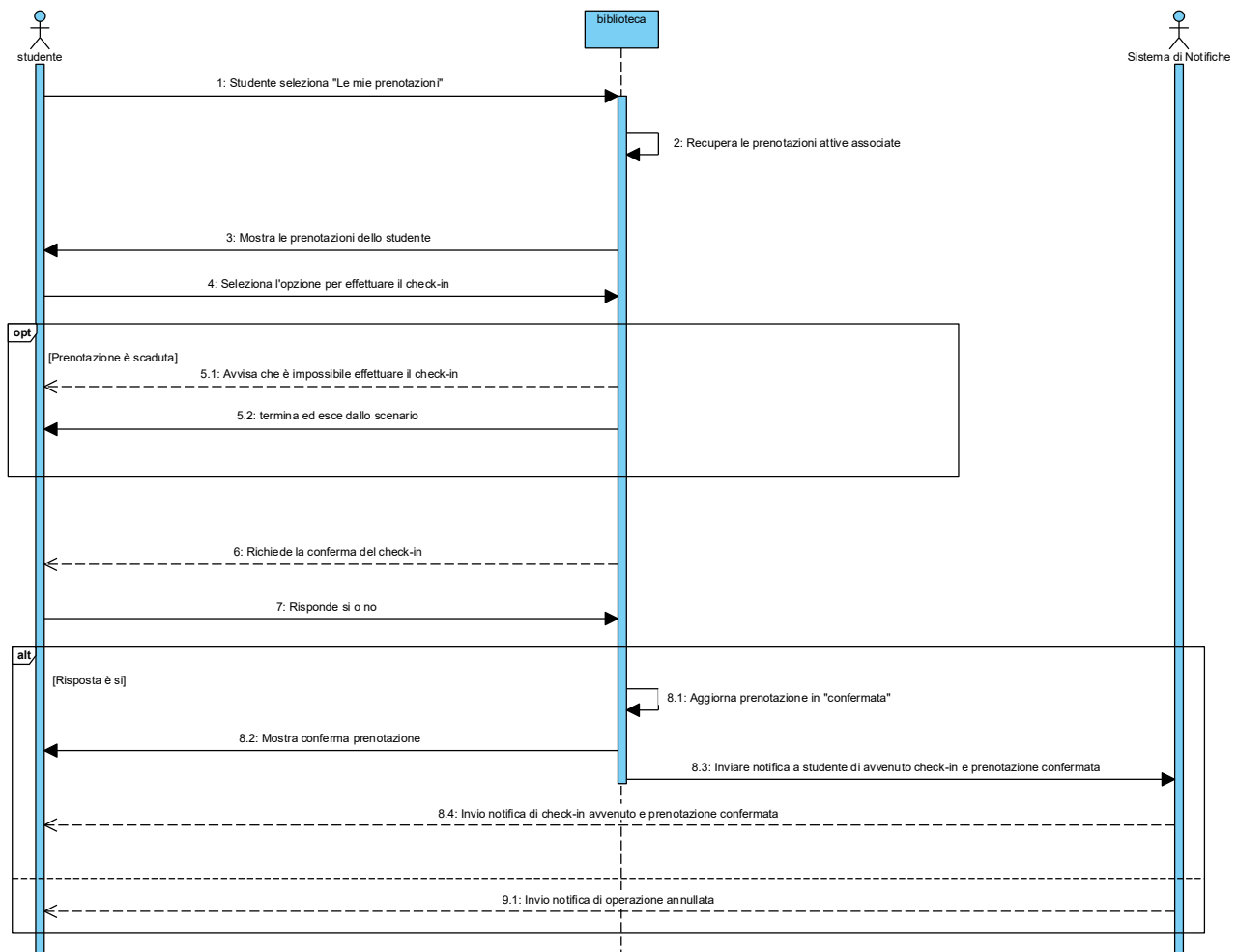


Figura 11 Sequence Diagram attore-sistema — EffettuaCheckIn

12. ScadenzaPrenotazione

Use Case	ScadenzaPrenotazione
ID	UC_12
Attori Primari	TimerDiSistema, Sistema di notifiche
Precondizioni	Esistono prenotazioni nello stato 'attiva'.
Postcondizioni	Le prenotazioni scadute vengono aggiornate allo stato 'scaduta'; le postazioni associate vengono liberate.
Scenario Base	1. TimerDiSistema avvia il controllo periodico delle prenotazioni. 2. SYSTEM recupera tutte le prenotazioni in stato "attiva". 3. for each prenotazione attiva 3.1. if l'intervallo di check-in è scaduto 3.1.1. SYSTEM aggiorna lo stato della prenotazione in "scaduta". 3.1.2. SYSTEM rende disponibile la postazione associata. 3.1.3. SYSTEM invoca Sistema di notifiche per inviare notifica di scadenza allo Studente coinvolto. - end if 3.2. if la prenotazione è prevista per il giorno successivo 3.2.1. SYSTEM invoca Sistema di notifiche per inviare notifica di promemoria allo Studente coinvolto. - end if - end for each
Requisiti coperti	RF_12, RF_21

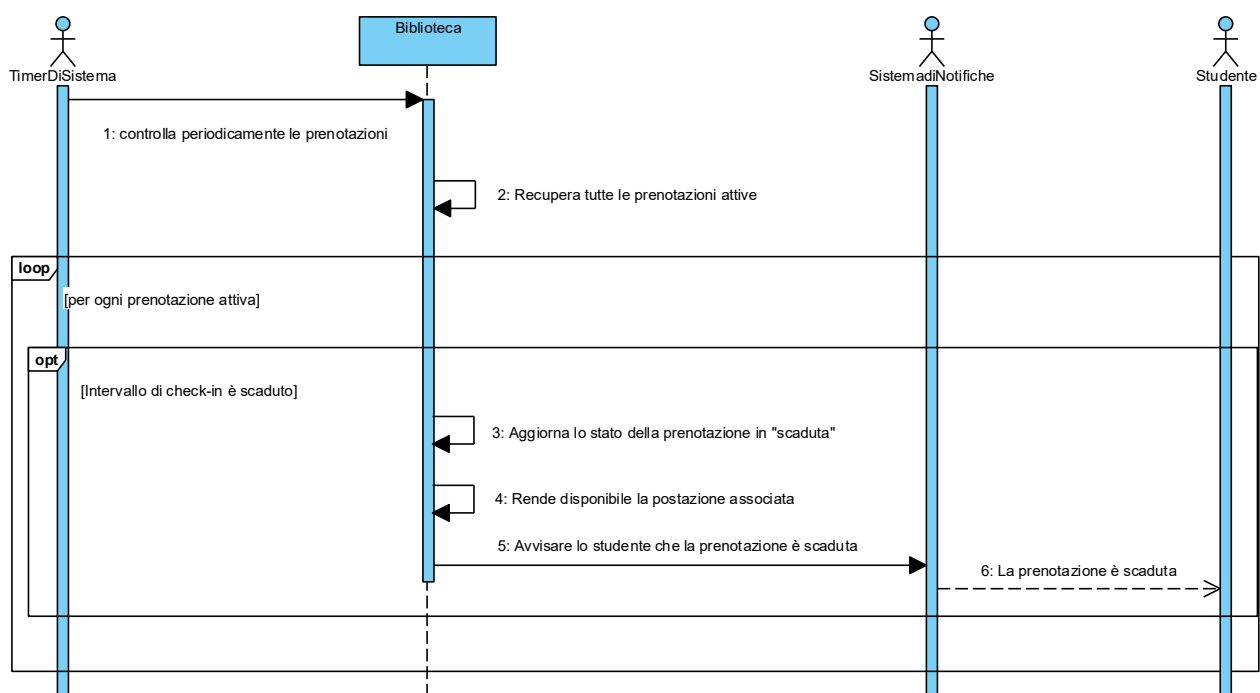


Figura 12 Sequence Diagram attore-sistema — ScadenzaPrenotazione

13. MonitoraggioAndamentoSale

Use Case	MonitoraggioAndamentoSale
ID	UC_13
Attori Primari	Bibliotecario
Precondizioni	L'utente è autenticato col ruolo di bibliotecario.
Postcondizioni	Nessun dato modificato.
Scenario Base	- Bibliotecario seleziona la schermata "Monitoraggio Sale". - SYSTEM recupera tutte le sale disponibili. - if non ci sono sale 3.1. SYSTEM segnala la mancanza di sale nel sistema. 3.2. exit Scenario Base. - end if - SYSTEM mostra l'elenco delle sale. - Bibliotecario seleziona la sala da monitorare. - SYSTEM recupera l'elenco delle prenotazioni attive per la sala selezionata. - SYSTEM calcola le statistiche per sala: numero di postazioni occupate e numero di postazioni libere. - SYSTEM calcola le statistiche aggregate: tasso di occupazione, numero di prenotazioni giornaliere e numero di prenotazioni non confermate tramite check-in. - SYSTEM mostra la dashboard con i dati calcolati e l'elenco delle prenotazioni attive con studente, fascia oraria e stato.
Requisiti coperti	RF_13, RF_14, RF_15, RF_18, RF_19, RF_20

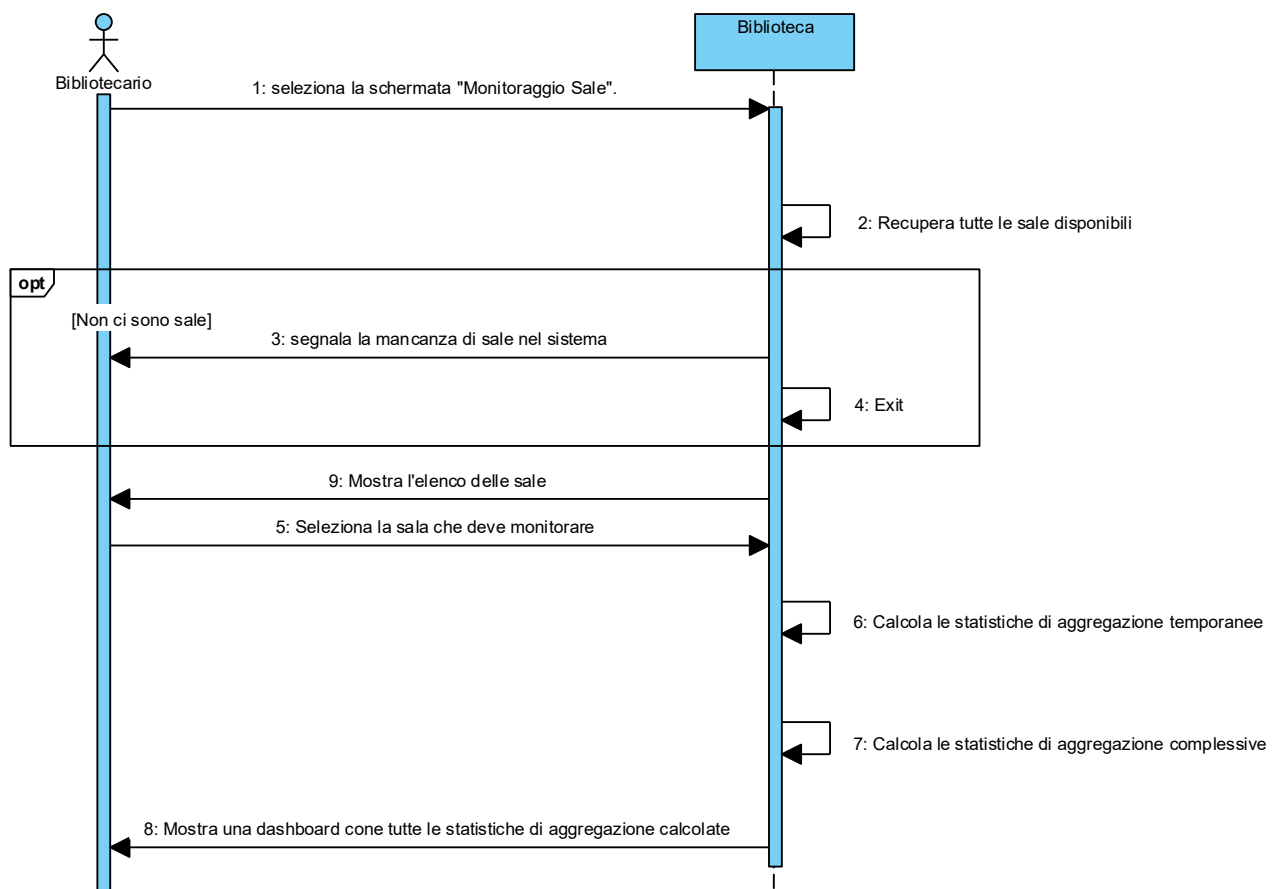


Figura 13 Sequence Diagram attore-sistema — MonitoraggioAndamentoSale

2.4. Matrice di Tracciabilità UC-Requisiti

La tabella seguente riporta la mappatura tra casi d'uso e requisiti funzionali coperti, a verifica della completezza della specifica.

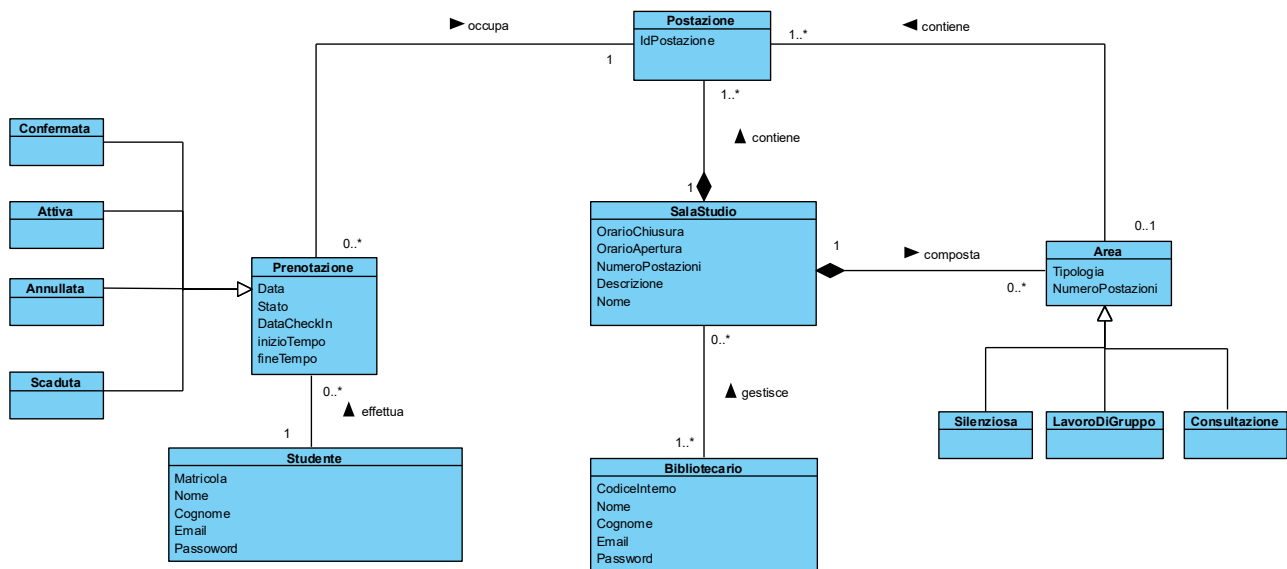
Caso d'Uso	UC_ID	Requisiti Coperti
RegistrazioneUtente	UC_01	RF_01
LoginUtente	UC_02	RF_02
CreaSala	UC_03	RF_03
ModificaSala	UC_04	RF_04
EliminaSala	UC_05	RF_05
VisualizzaStoricoPrenotazioni	UC_06	RF_06, RF_16
ConsultaFasceDisponibili	UC_07	RF_07, RF_24
EffettuaPrenotazione	UC_08	RF_08, RF_23, RF_24
VisualizzaStoricoPersonale	UC_09	RF_09, RF_17
AnnullaPrenotazione	UC_10	RF_10, RF_22
EffettuaCheckIn	UC_11	RF_11
ScadenzaPrenotazione	UC_12	RF_12, RF_21
MonitoraggioAndamentoSale	UC_13	RF_13, RF_14, RF_15, RF_18, RF_19, RF_20

2.5. Domain Model

Il Domain Model rappresenta i concetti del dominio del problema e le loro relazioni, indipendentemente da qualsiasi considerazione implementativa o progettuale. Non contiene stereotipi GRASP/GoF né classi software (come controller, cataloghi o GestorePersistenza): modella esclusivamente le entità e le relazioni del dominio reale, così come emergono dall'analisi testuale della specifica informale.

Il diagramma identifica le seguenti entità e i relativi attributi principali:

- **Studente** (*Matricola, Nome, Cognome, Email, Password*) e **Bibliotecario** (*CodiceInterno, Nome, Cognome, Email, Password*) come ruoli utente del sistema.
- **SalaStudio** (*Nome, Descrizione, NumeroPostazioni, OrarioApertura, OrarioChiusura*) come entità che rappresenta uno spazio fisico della biblioteca gestito da un Bibliotecario.
- **Area** (*Tipologia, NumeroPostazioni*) come suddivisione opzionale di una SalaStudio, con tre sottotipi concreti: Silenziosa, LavoroDiGruppo e Consultazione.
- **Postazione** (*IdPostazione*) come unità fisica prenotabile, contenuta in una SalaStudio e opzionalmente afferente a un'Area.
- **Prenotazione** (*Data, Stato, DataCheckIn, inizioTempo, fineTempo*) come entità centrale che associa uno Studente a una Postazione per una specifica fascia oraria.



Le principali relazioni nel Domain Model sono, con le rispettive molteplicità:

- Bibliotecario gestisce 0..* SalaStudio (relazione di gestione).
- SalaStudio è composta da 0..* Area (suddivisione opzionale in un'unica area tipizzata).
- SalaStudio contiene 1..* Postazione.
- Area contiene 1..* Postazione (le postazioni dell'area sono un sottoinsieme di quelle della SalaStudio).
- Studente effettua 0..* Prenotazione.
- Prenotazione occupa 1 Postazione (la fascia oraria è rappresentata dagli attributi inizioTempo e fineTempo della Prenotazione stessa, non come entità separata).
- Prenotazione ha uno stato: Attiva, Annullata, Confermata, Scaduta.

Scelta progettuale — Disponibilità delle postazioni

Si è scelto di non modellare la disponibilità di una Postazione come attributo statico persistente. La disponibilità viene invece calcolata dinamicamente a runtime, verificando l'assenza di Prenotazioni attive (stato Attiva o Confermata) per la stessa Postazione nella medesima fascia oraria e data. Questo approccio garantisce consistenza senza ridondanza dei dati e semplifica il modello di dominio, eliminando la necessità di aggiornare esplicitamente uno stato di occupazione ad ogni variazione.

2.6. Domain Model con Responsabilità GRASP

Il Domain Model con Responsabilità GRASP estende il Domain Model puro introducendo le responsabilità di progetto secondo i principi GRASP (*General Responsibility Assignment Software Patterns*). Rispetto al Domain Model, aggiunge le classi software necessarie per il design del sistema e assegna a ciascuna entità e classe software una responsabilità specifica. Questo modello costituisce il ponte tra l'analisi del dominio e l'architettura implementata descritta nella Sezione 3.

Introduzione dei Controller — Use Case Controller (GRASP)

I controller `GestorePrenotazioni`, `GestoreSaleStudio` e `GestoreAccesso` vengono introdotti come Use Case Controller secondo il principio GRASP omonimo. Il loro ruolo è ricevere e coordinare le operazioni di sistema innescate dagli attori, delegando il lavoro alle entità appropriate. La suddivisione per dominio funzionale (prenotazioni, sale, accesso) garantisce High Cohesion e riduce il Coupling tra le classi.

Introduzione dei Cataloghi — Information Expert (GRASP)

I cataloghi `CatalogoPrenotazioni` e `CatalogoSaleStudio` vengono introdotti come Information Expert delle rispettive collezioni globali. Secondo il principio GRASP dell'Information Expert, la responsabilità di una computazione deve essere assegnata alla classe che possiede le informazioni necessarie per eseguirla. Poiché le operazioni di ricerca e filtraggio delle prenotazioni o delle sale richiedono l'accesso all'intera collezione, introdurre un catalogo dedicato evita di distribuire questa responsabilità tra i controller o le entità singole.

Postazione come Information Expert della propria disponibilità

Postazione viene annotata come Information Expert della propria disponibilità. Poiché è l'entità che possiede le informazioni sulle prenotazioni ad essa associate (tramite la relazione con `Prenotazione`), è la candidata naturale a rispondere alla domanda "questa postazione è disponibile in questa fascia oraria?". Il calcolo rimane dinamico — non c'è un attributo di stato persistito — ma la responsabilità è localizzata in `Postazione`, coerentemente con il principio GRASP.

Prenotazione come State Dependent — State Pattern (GoF)

`Prenotazione` viene modellata come State Dependent perché il suo comportamento varia in funzione dello stato corrente (`Attiva`, `Confermata`, `Annullata`, `Scaduta`). Operazioni come il check-in, l'annullamento o la scadenza hanno semantica e precondizioni diverse a seconda dello stato in cui si trova la prenotazione. Applicare il pattern State (GoF) permette di incapsulare il comportamento specifico di ciascuno stato in classi dedicate (`StatoPAttiva`, `StatoPConfermata`, `StatoPAnnullata`, `StatoPScaduta`), evitando la proliferazione di condizionali `if/switch` nell'entità `Prenotazione` e rendendo il ciclo di vita estendibile senza modificare la classe principale.

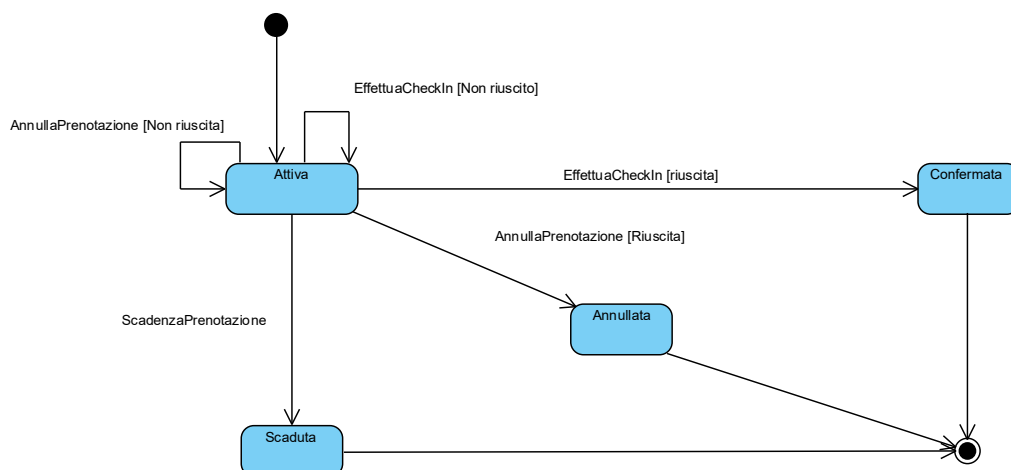
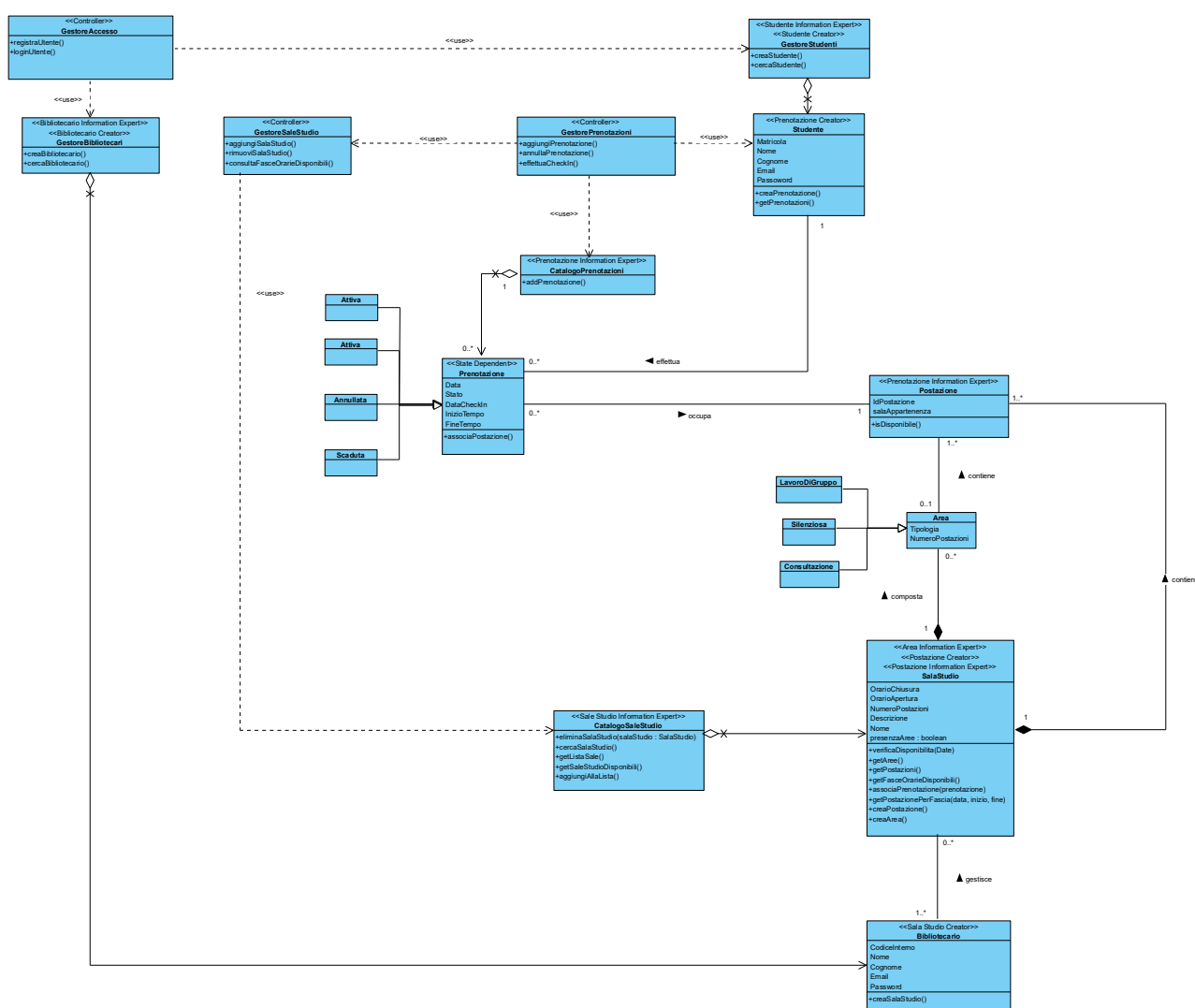


Figura 14 State machine diagram -- Classe Prenotazione



2.7. Diagrammi di Sequenza con Responsabilità

In questa sezione vengono riportati i diagrammi di sequenza con responsabilità per i casi d'uso implementati. A differenza dei diagrammi attore-sistema presentati in § 2.3 — che modellano l'interazione tra l'attore e il sistema come scatola nera — i diagrammi di questa sezione mostrano la distribuzione delle responsabilità tra i componenti software, applicando i principi GRASP descritti in § 2.6.

RegistrazioneUtente

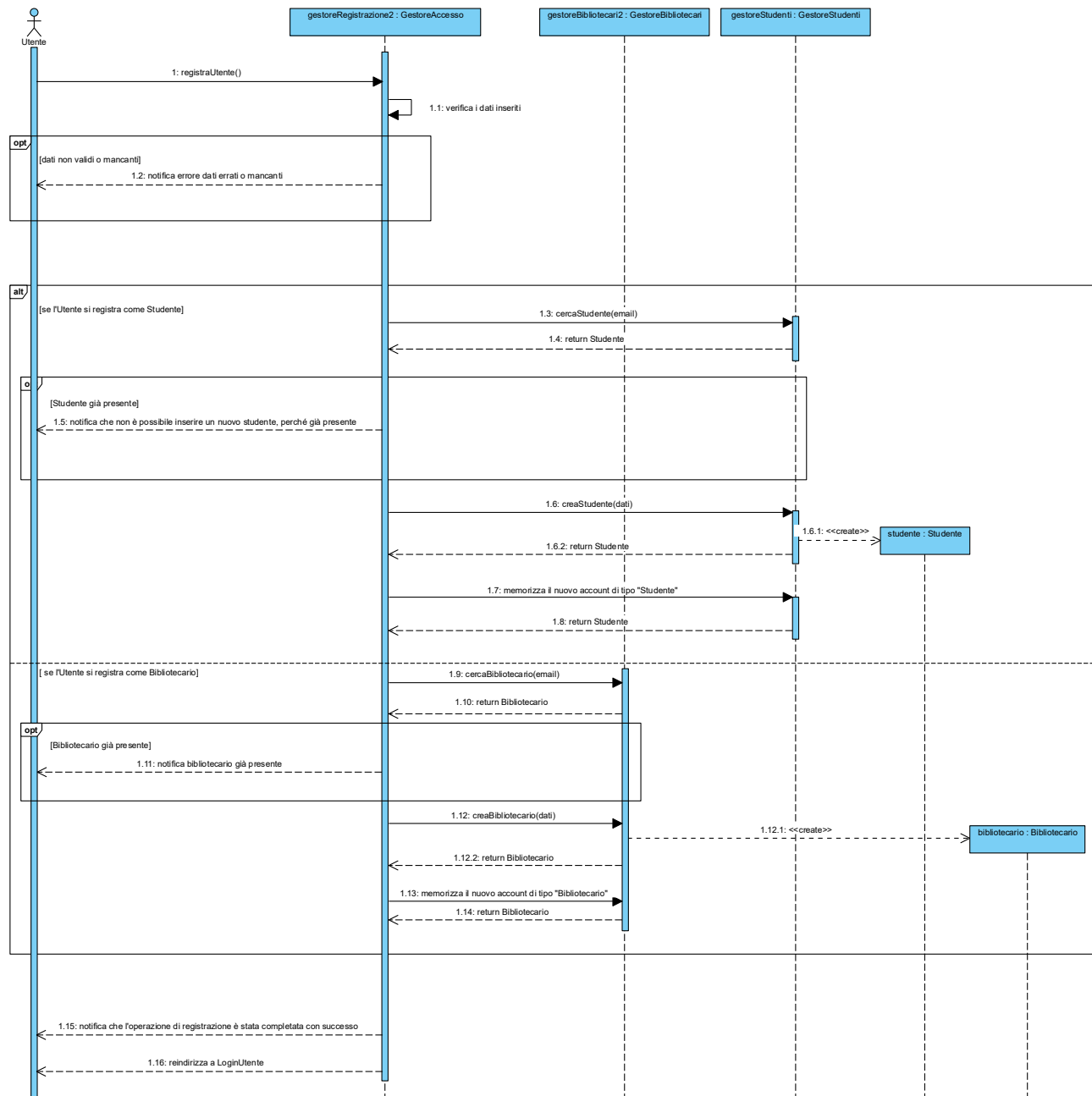


Figura 15 Sequence diagram attore-sistema con responsabilità — RegistrazioneUtente

LoginUtente

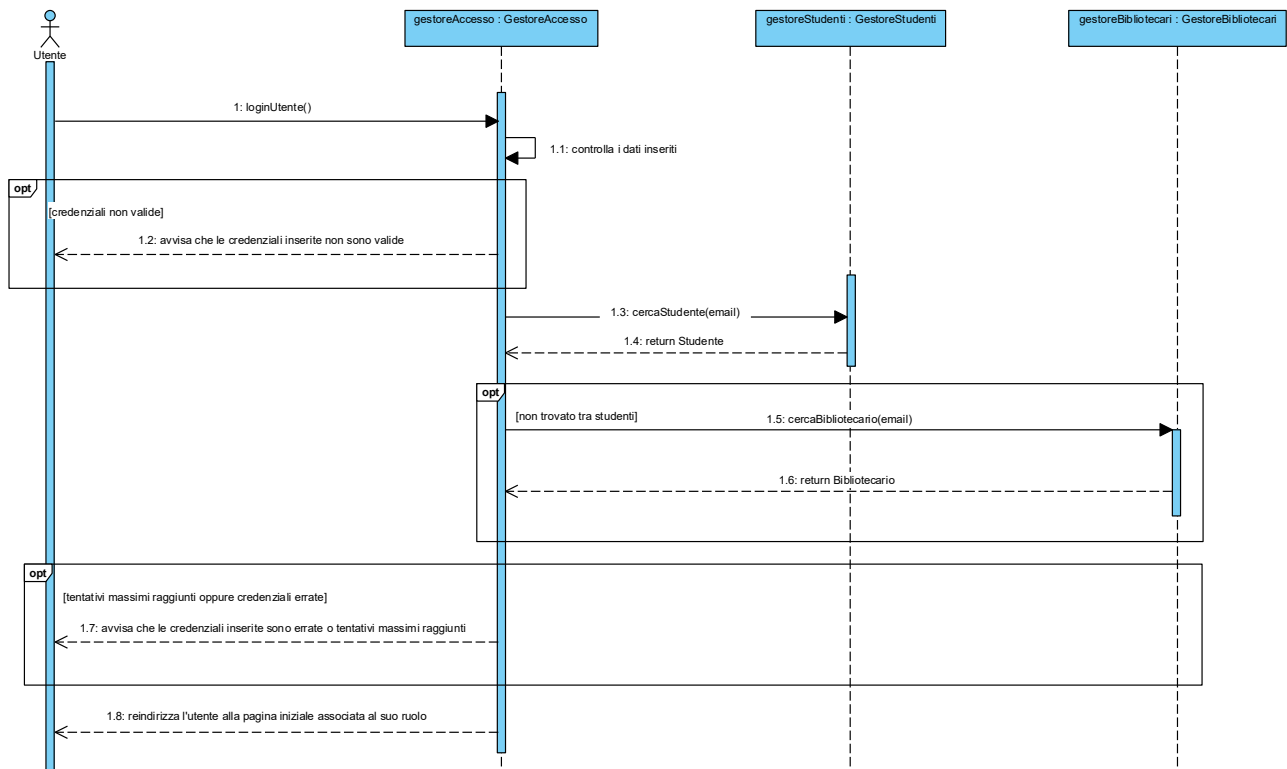


Figura 16 Sequence diagram attore-sistema con responsabilità — LoginUtente

CreaSala

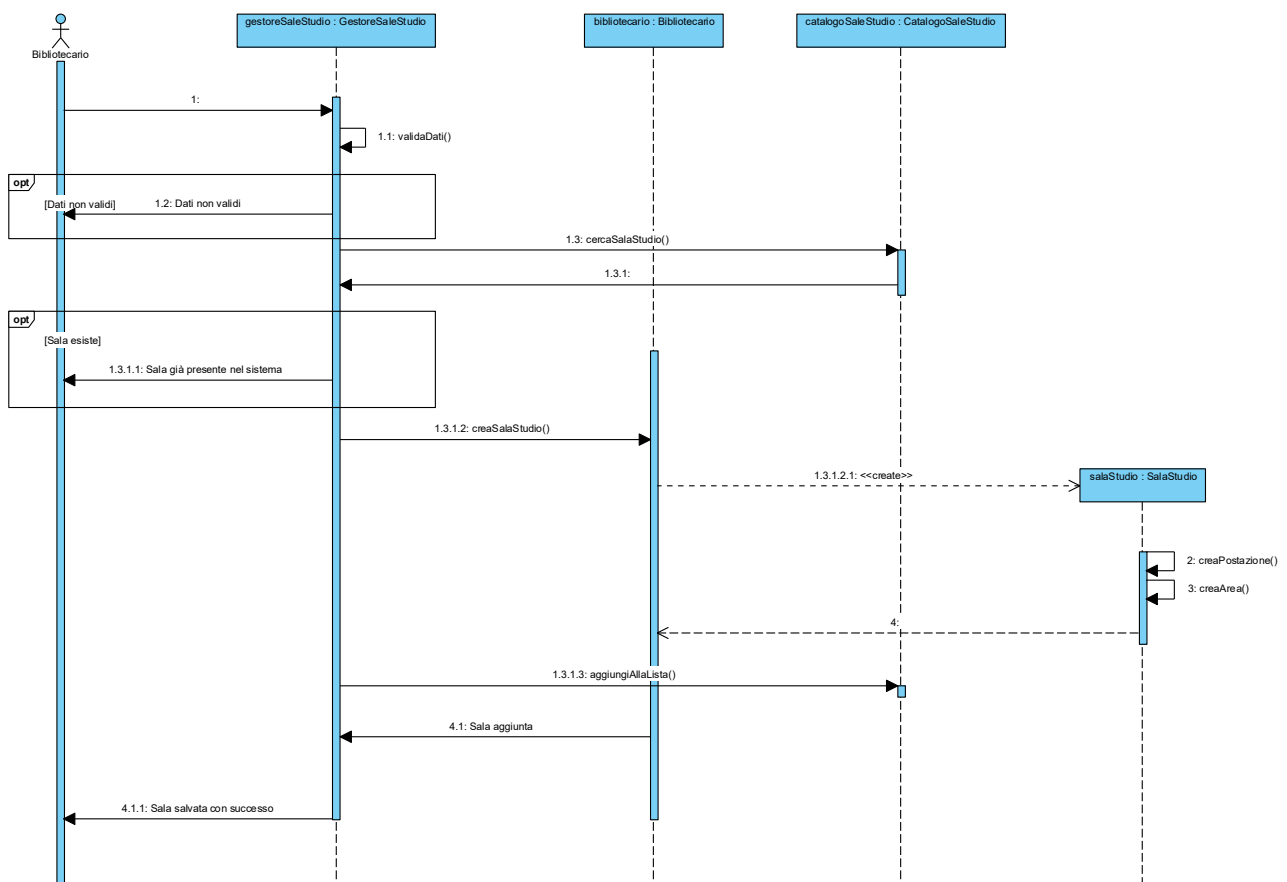


Figura 17 Sequence diagram attore-sistema con responsabilità — CreaSala

ConsultaFasceDisponibili

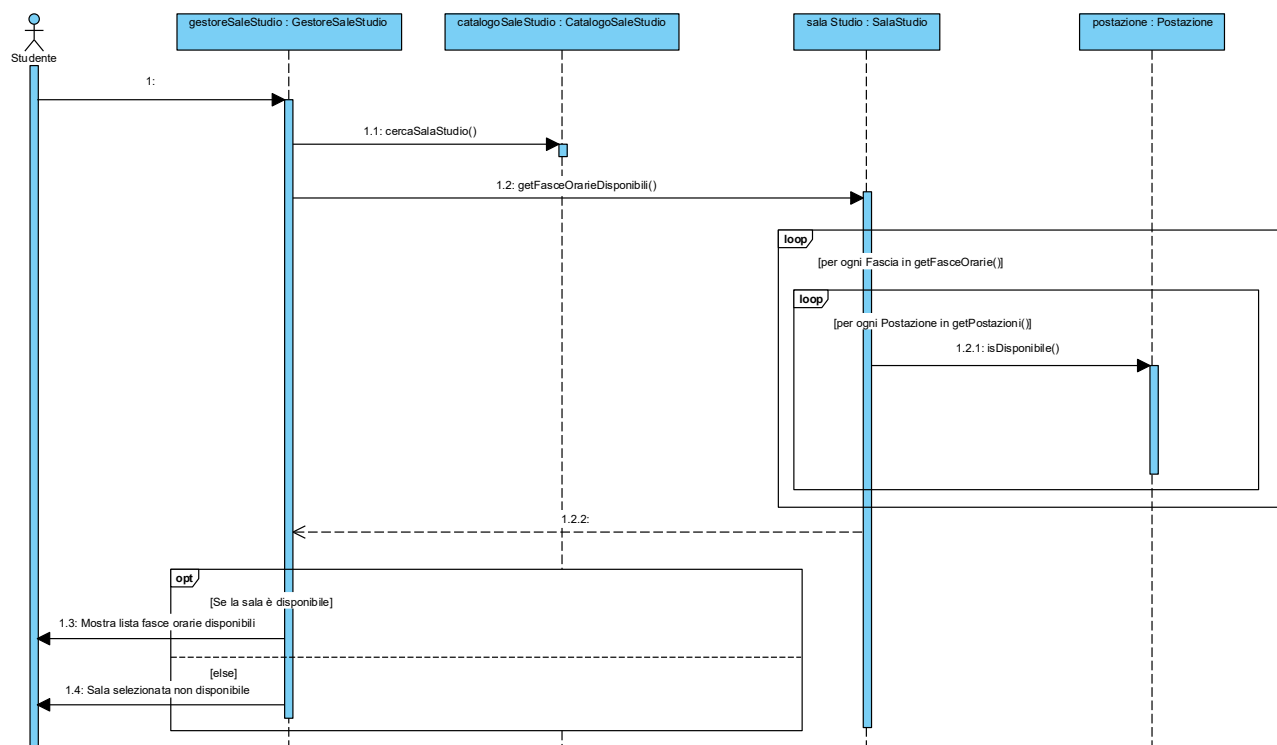


Figura 18 Sequence diagram attore-sistema con responsabilità — ConsultaFasceDisponibili

EffettuaCheckIn

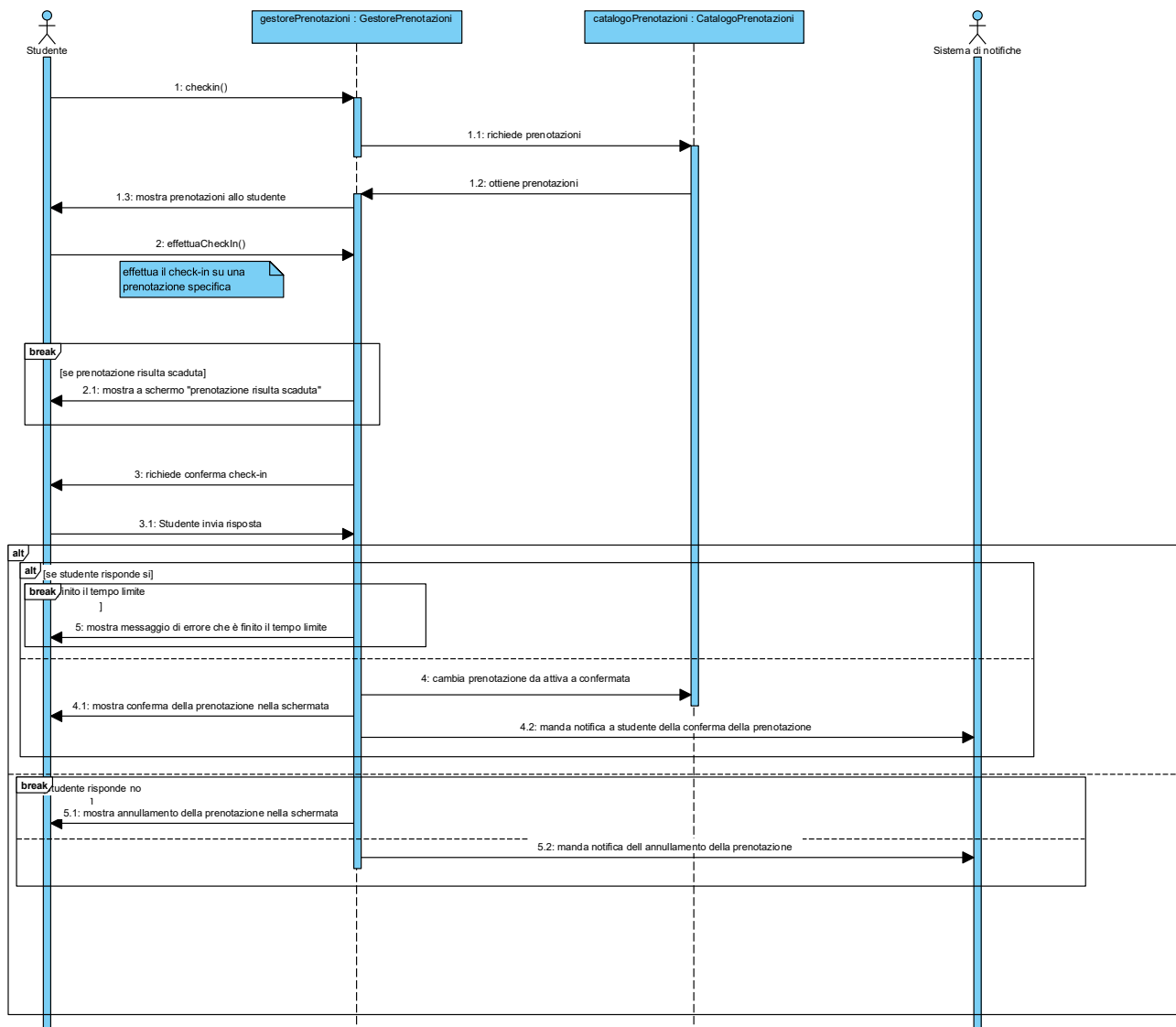


Figura 19 Sequence diagram attore-sistema con responsabilità — EffettuaCheckIn

EffettuaPrenotazione

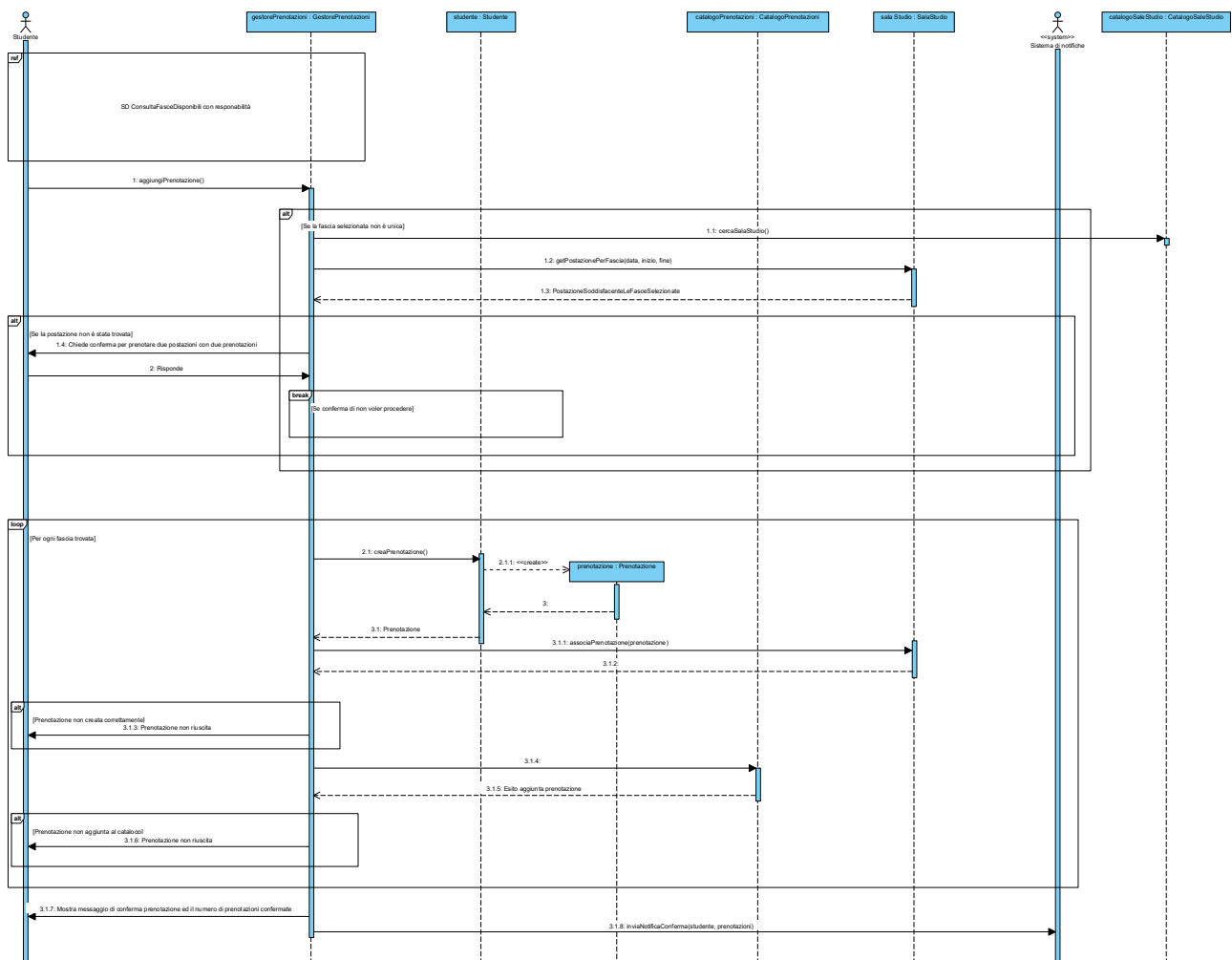


Figura 20 Sequence diagram attore-sistema con responsabilità — EffettuaPrenotazione

3. Design del Sistema Software Implementato

In questa sezione vengono descritti i modelli di design che documentano l'architettura del sistema e la dinamica di esecuzione dei casi d'uso realizzati. Il sistema segue il pattern architetturale BCED (Boundary-Control-Entity-Database) e applica sistematicamente i principi GRASP.

3.1. Architettura Generale

Il sistema adotta un'architettura a due livelli (two-tier): un livello applicativo (client Java/Swing) e un livello dati (MySQL). Lo stack tecnologico è il seguente:

- Linguaggio: Java 26
- Interfaccia grafica: Java Swing (componenti custom e IntelliJ GUI Designer per le form principali)
- ORM: Hibernate 6.x con JPA (schema auto-generato tramite hbm2ddl)
- Database: MySQL — schema biblioteca_db
- Notifiche email: Jakarta Mail 2.0.1 (com.sun.mail:jakarta.mail) con GreenMail come fake server SMTP embedded per i test
- Build tool: Maven (artifactId: Locus, groupId: it.unina.locus)

3.2. Pattern Architetturale BCED

Il sistema implementa il pattern architetturale BCED (Boundary-Control-Entity-Database):

- Boundary: le classi Form* (FormStudente, FormBibliotecario, FormLogin, FormRegistrazione, FormCheckIn, FormConsultaFasceOrarie, FormEffettuaPrenotazione, FormCreaSala, ecc.) rappresentano il livello di presentazione.
- Control: i Gestori (GestoreAccesso, GestorePrenotazioni, GestoreSaleStudio, GestoreStudenti, GestoreBibliotecari) fungono da Use Case Controller GRASP. GestorePersistenza funge da Facade verso il livello ORM.
- Entity: le classi Studente, Bibliotecario, SalaStudio, Area, Postazione, Prenotazione sono entità di dominio persistite via Hibernate con annotazione @Entity.
- Database: il livello di persistenza è gestito da GestorePersistenza e JpaUtil, che incapsulano l'EntityManagerFactory e le transazioni JPA.

3.3. Design Pattern Adottati

3.3.1. GRASP — Assegnazione delle Responsabilità

I controller sono suddivisi per responsabilità funzionale (non per attore), in accordo col principio di High Cohesion / Low Coupling:

- GestoreAccesso: responsabile di login e registrazione.
- GestorePrenotazioni: responsabile del ciclo di vita delle prenotazioni (effettuazione, check-in, annullamento, scadenza).
- GestoreSaleStudio: responsabile della creazione, modifica e eliminazione delle sale e del monitoraggio.

Gli Information Expert sono assegnati coerentemente:

- CatalogoSaleStudio: IE della collezione globale delle SalaStudio.
- CatalogoPrenotazioni: IE della collezione globale delle Prenotazione.

- **SalaStudio**: IE della propria struttura e generatore delle FasciaOraria (slot da 1 ora, generati dinamicamente da inizioApertura/fineApertura con LocalTime).
- **Postazione**: IE della propria disponibilità (calcolata dinamicamente, non persistita).
- **Prenotazione**: IE del proprio stato e dei propri attributi temporali.

Il ciclo di vita della Prenotazione (Attiva → Confermata / Annullata / Scaduta) è modellato con il pattern State (GoF):

- **StatoPrenotazione**: interfaccia del pattern State.
- **StatoPAttiva**, **StatoPConfermata**, **StatoPAnnullata**, **StatoPScaduta**: ConcreteState.

Per la persistenza Hibernate, lo stato è salvato come `@Enumerated(EnumType.STRING)` tramite `StatoPrenotazioneEnum`. L'oggetto-stato comportamentale è `@Transient` e viene ricostruito dall'enum tramite `@PostLoad`.

3.3.3. Singleton Pattern (GoF) — Sessione Utente

La classe Sessione implementa il Singleton GoF: mantiene il riferimento all'utente autenticato corrente (Studente o Bibliotecario). Invariante: al più uno tra i due è valorizzato; l'apertura di una sessione per un ruolo azzerà l'altro.

3.3.4. Façade Pattern (GoF) — GestorePersistenza

GestorePersistenza espone un'interfaccia semplificata verso il livello JPA/Hibernate, nascondendo `JpaUtil` e `EntityManager`. I Sequence Diagram si fermano ai metodi pubblici di GestorePersistenza; le chiamate JPA interne non compaiono come lifeline.

3.4. Modello delle Classi — System Model

Il diagramma delle classi di sistema documenta l'architettura completa con stereotipi GRASP e GoF, relazioni di dipendenza e associazione, attributi e metodi principali.

La tabella seguente riassume le classi principali e le loro responsabilità:

Classe	Stereotipo	Responsabilità
GestoreAccesso	Controller (GRASP), Façade	Use Case Controller per autenticazione e registrazione. Gestisce login (con blocco dopo numero massimo tentativi falliti), registrazione e validazione con accesso alla persistenza.
GestorePrenotazioni	Controller (GRASP), Façade	Use Case Controller responsabile del ciclo di vita delle prenotazioni: effettuazione, check-in e consultazione. Orchestra <code>CatalogoSaleStudio</code> e <code>CatalogoPrenotazioni</code> .
GestoreSaleStudio	Controller (GRASP), Façade	Use Case Controller per la gestione delle sale: creazione, modifica, eliminazione e monitoraggio.
GestorePersistenza	Façade	Facade verso il livello di persistenza Hibernate/JPA. Punto di accesso unico alle operazioni CRUD; incapsula <code>JpaUtil</code> e <code>EntityManager</code> .
CatalogoSaleStudio	Information Expert (GRASP)	Information Expert dell'intera collezione delle SalaStudio. Gestisce il reperimento di sale, postazioni e fasce orarie disponibili.

CatalogoPrenotazioni	Information Expert (GRASP)	Information Expert dell'intera collezione delle Prenotazione. Gestisce registrazione, aggiornamento e recupero filtrato delle prenotazioni dalla persistenza.
SalaStudio	Entity Bean, ORM Persistable	Entità di dominio. Information Expert della propria struttura: aggrega Postazione e Area. Genera FasciaOraria dinamicamente dagli orari di apertura e chiusura.
Postazione	Entity Bean, ORM Persistable	Entità di dominio. Information Expert della propria disponibilità, derivata dinamicamente dalle prenotazioni esistenti valutate per data e fascia oraria.
Area	Entity Bean, ORM Persistable	Entità di dominio che rappresenta una suddivisione opzionale di una SalaStudio (es. area silenziosa, consultazione, lavoro di gruppo).
Prenotazione	Entity Bean, ORM Persistable	Entità di dominio. Information Expert del proprio stato. Ciclo di vita modellato con State pattern; stato persistito come @Enumerated(EnumType.STRING), oggetto-stato comportamentale @Transient e ricostruito tramite @PostLoad.
StatoPrenotazione	State (GoF)	Interfaccia del pattern State per la gestione del ciclo di vita della Prenotazione.
StatoPAttiva / StatoPAnnullata / StatoPConfermata / StatoPScaduta	ConcreteState (GoF)	Implementazioni concrete dello State per ciascun stato del ciclo di vita della prenotazione.
FasciaOraria	DTO	Data Transfer Object immutabile che rappresenta una fascia oraria. Generata dinamicamente da SalaStudio; non persistita.
Sessione	Singleton (GoF)	Singleton che mantiene il riferimento all'utente autenticato corrente (Studente o Bibliotecario). Invariante: al più uno tra i due è valorizzato.
Studente	Entity Bean, ORM Persistable	Entità di dominio. Information Expert dei propri dati anagrafici e delle prenotazioni associate. Matricola come chiave naturale.
Bibliotecario	Entity Bean, ORM Persistable	Entità di dominio. Creator delle sale studio. Codice interno come chiave naturale.
ServizioNotifiche	Boundary	Boundary verso il sistema di posta esterno. Incapsula configurazione SMTP e invio notifiche email via Jakarta Mail.
GestoreStudenti / GestoreBibliotecari	Controller (GRASP)	Controller specializzati per le operazioni sui profili di studenti e bibliotecari.

3.5. Snippet di Codice — Scelte Progettuali Chiave

3.5.1. Disponibilità Postazione — Calcolo Dinamico

La disponibilità di una Postazione non è un attributo persistito ma viene calcolata dinamicamente dalle prenotazioni attive esistenti per data e fascia oraria, garantendo consistenza senza ridondanza dati e con il vincolo di unicità sulla tupla (Postazione, FasciaOraria, data) con stato='attiva' gestito a livello di database.

3.5.2. Generazione FasciaOraria

Le FasciaOraria non sono entità persistite. Vengono generate dinamicamente da SalaStudio come slot orari a partire da inizioApertura fino a fineApertura (LocalTime), usando la classe FasciaOraria come DTO immutabile (campi final). Questo ne consente l'uso sicuro come chiave in mappe di disponibilità.

3.5.3. Invio Email via Jakarta Mail

ServizioNotifiche incapsula la configurazione SMTP e l'invio delle notifiche email via Jakarta Mail 2.0.1. I parametri di connessione vengono letti da un file di configurazione esterno. GreenMail viene utilizzato come server SMTP embedded nei test di integrazione.

3.6. Diagrammi di Sequenza con architettura BCED

I diagrammi di sequenza BCED modellano la dinamica di esecuzione a livello architetturale, mostrando l'interazione tra le classi Boundary, Control, Entity e Database per i casi d'uso implementati. I messaggi riportano i metodi effettivi delle classi coinvolte; il punto di arresto è GestorePersistenza, che funge da Façade verso il livello JPA/Hibernate — le chiamate interne all'EntityManager non compaiono come lifeline. La catena principale di delega è: Form* (Boundary) → Gestore* (Control) → Catalogo* / Entità (Entity) → GestorePersistenza (Database).

RegistrazioneUtente

Controller: GestoreAccesso. Il diagramma mostra la catena di messaggi dal form boundary fino a GestorePersistenza.

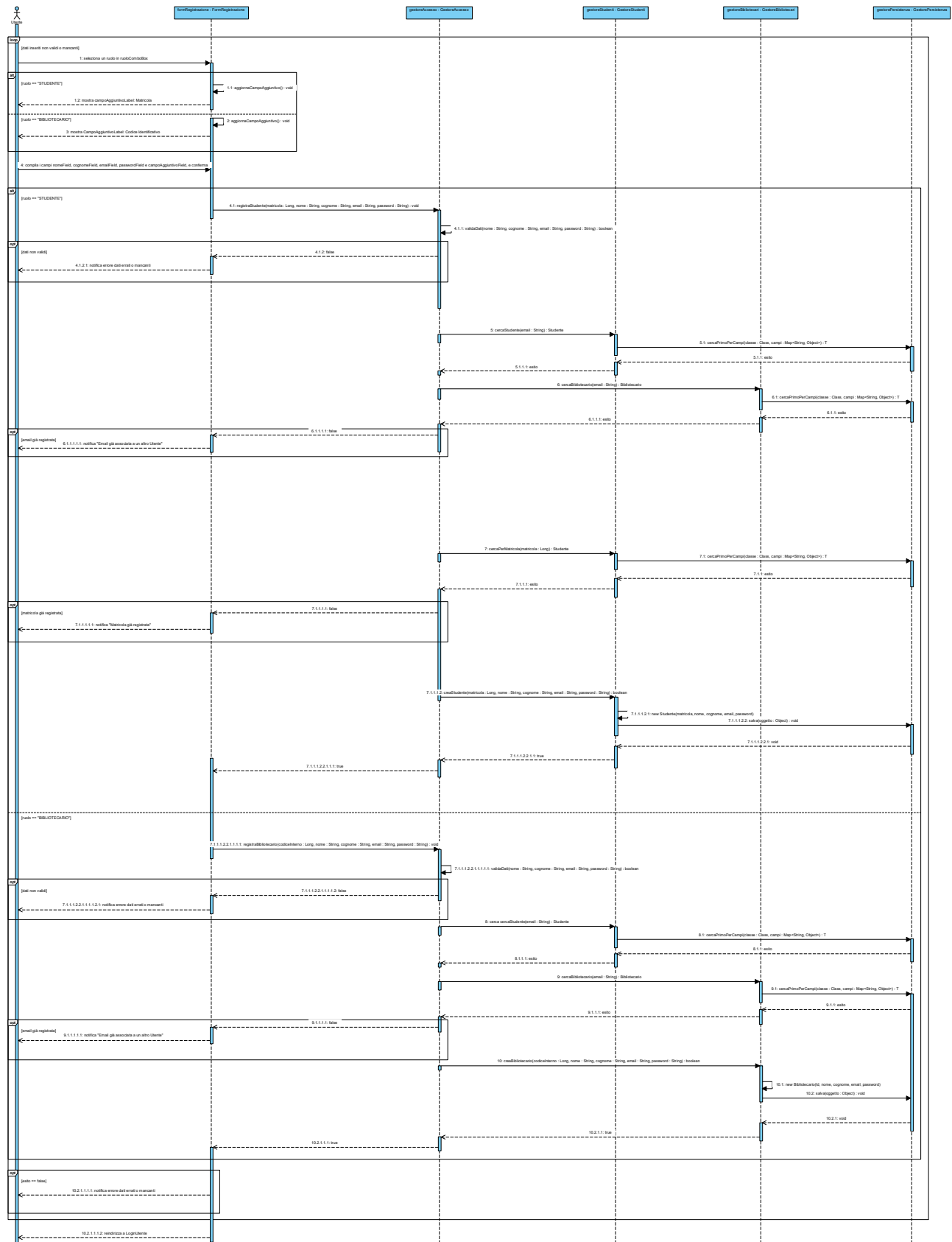


Figura 21 Sequence diagram con responsabilità BCED -- RegistrazioneUtente

LoginUtente

Controller: GestoreAccesso. Il diagramma mostra la catena di messaggi dal form boundary fino a GestorePersistenza.

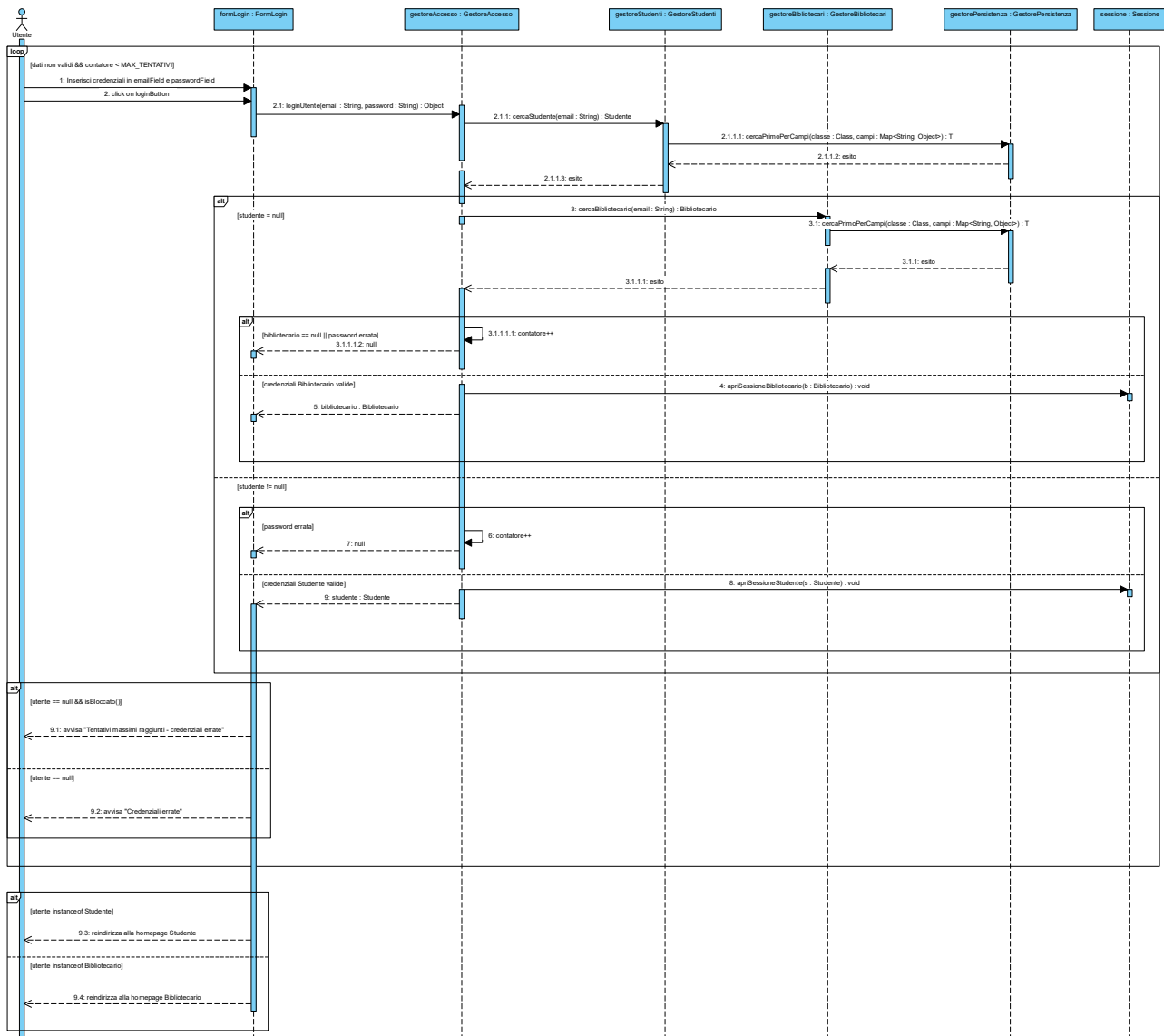


Figura 22 Sequence diagram con responsabilità BCED -- LoginUtente

CreaSala

Controller: GestoreSaleStudio. Il diagramma mostra la catena di messaggi dal form boundary fino a GestorePersistenza.

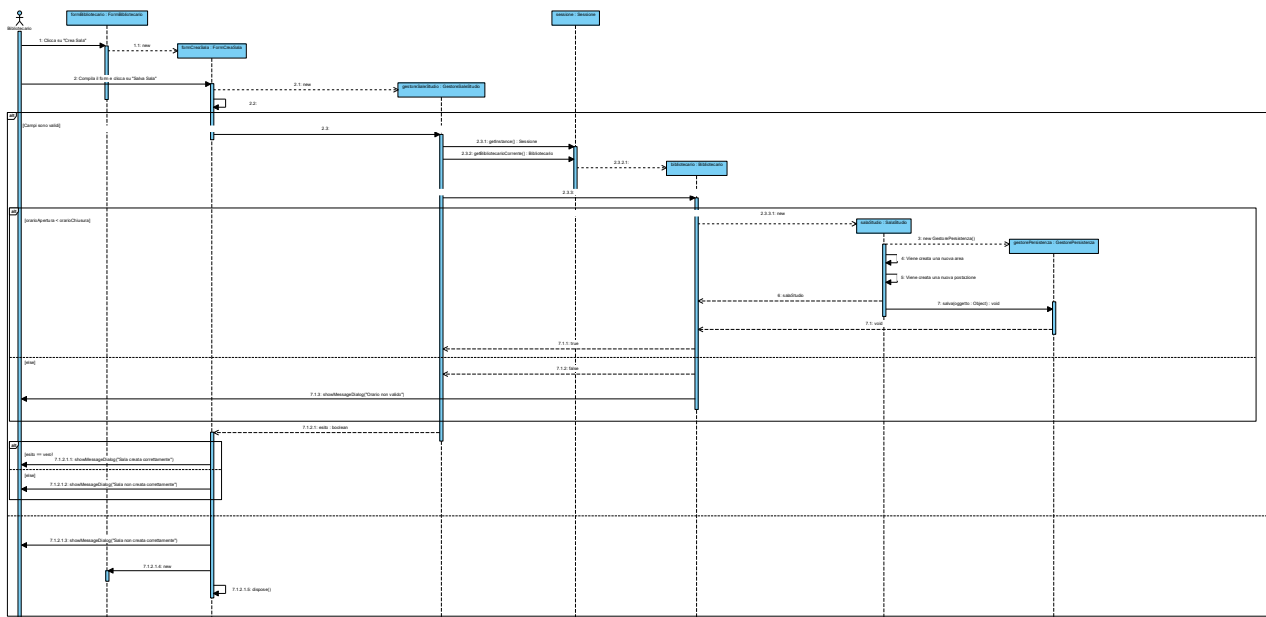


Figura 23 Sequence diagram con responsabilità BCED -- CreaSala

ConsultaFasceDisponibili

Controller: `GestoreSaleStudio`. Il diagramma mostra la catena di messaggi dal form boundary fino a `GestorePersistenza`.

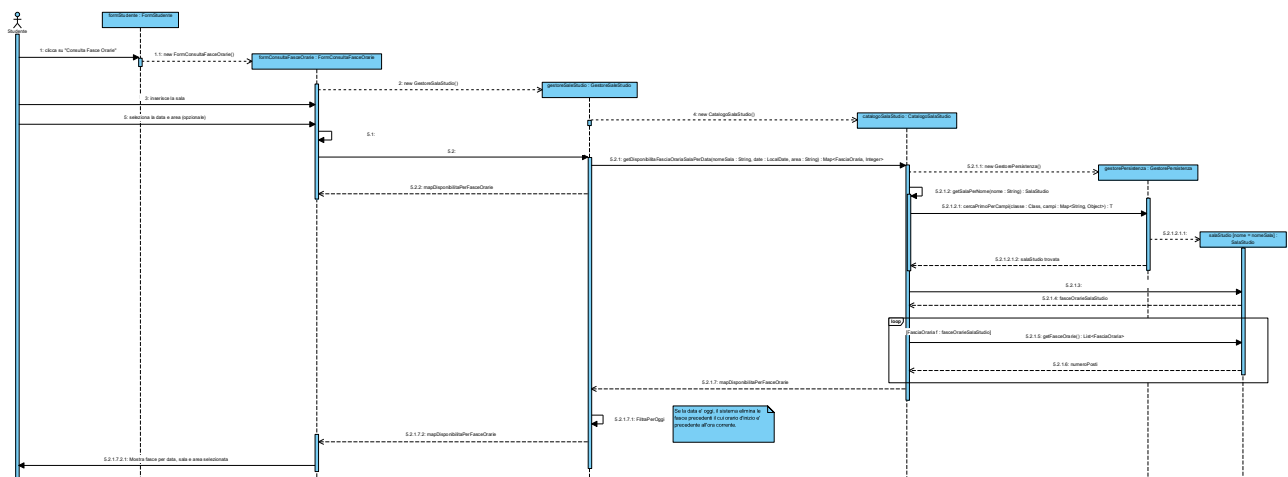


Figura 24 Sequence diagram con responsabilità BCED -- ConsultaFasceDisponibili

EffettuaPrenotazione

Controller: GestorePrenotazioni. Il diagramma mostra la catena di messaggi dal form boundary fino a GestorePersistenza.

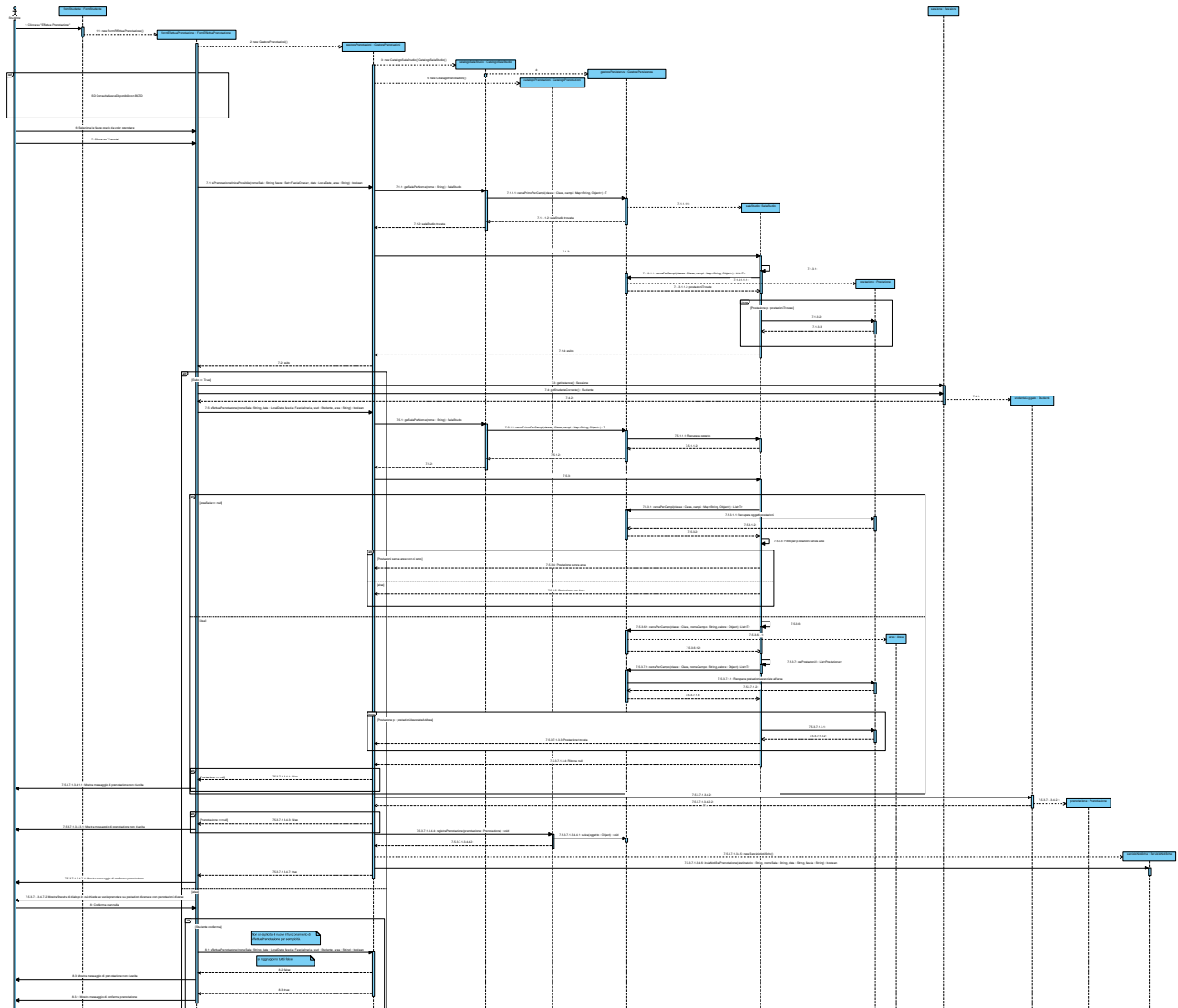


Figura 25 Sequence diagram con responsabilità BCED -- EffettuaPrenotazione

EffettuaCheckIn

Controller: GestorePrenotazioni. Il diagramma mostra la catena di messaggi dal form boundary fino a GestorePersistenza.

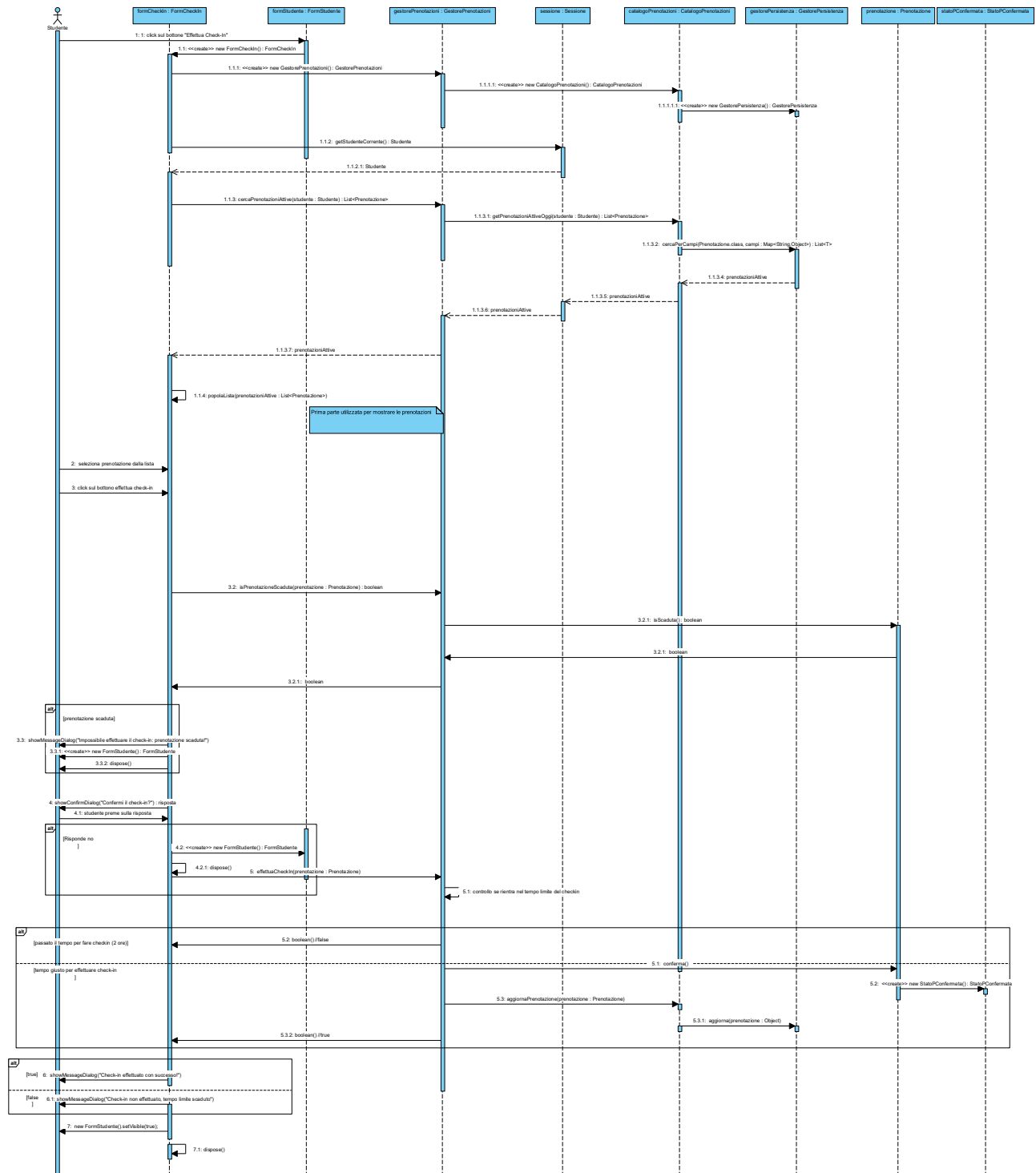
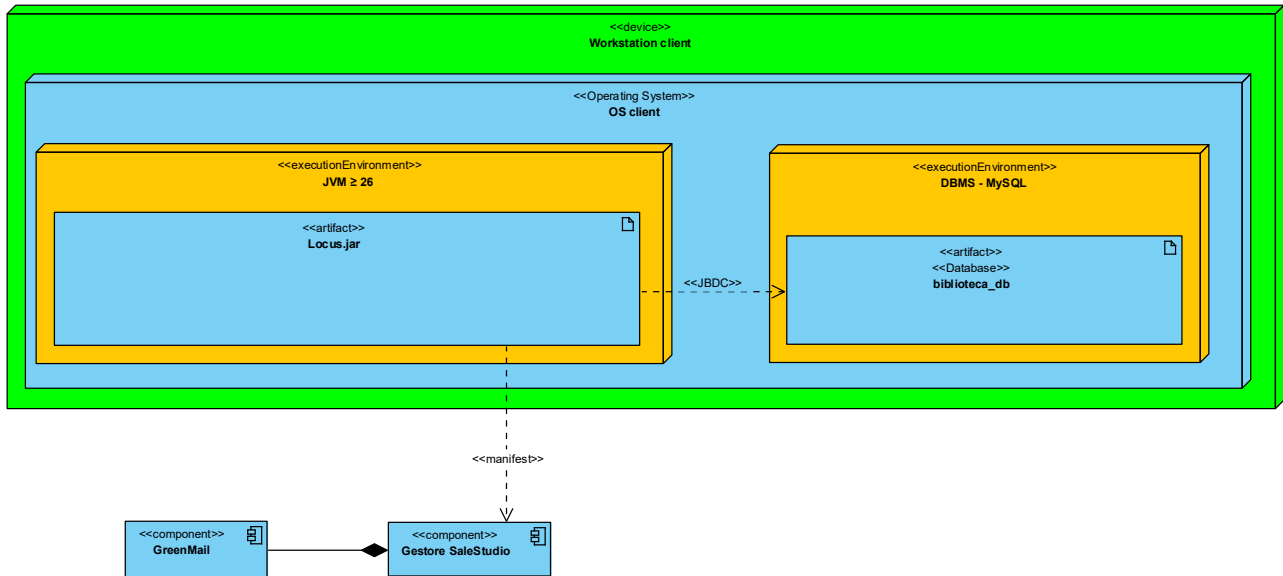


Figura 26 Sequence diagram con responsabilità BCED -- EffettuaCheckIn

4. Diagramma di Deploy

Il diagramma di deploy illustra l'ambiente di esecuzione, i nodi coinvolti e la distribuzione dei principali componenti applicativi. Il sistema adotta un'architettura two-tier: un nodo Workstation (client) e un nodo Database (MySQL), co-localizzabili sulla stessa macchina.



4.1. Nodi e Componenti

Il nodo Workstation ospita:

- JVM ≥ 26 come execution environment.
- bibliotecaSys.jar — l'artefatto applicativo principale prodotto da Maven.
- GreenMail (componente SMTP embedded) — utilizzato in fase di test come server di posta in-process.
- Jakarta Mail (com.sun.mail:jakarta.mail 2.0.1) — libreria per l'invio email in produzione.

Il nodo DBMS - MySQL ospita:

- MySQL come execution environment (stereotipo «executionEnvironment»).
- biblioteca_db — l'istanza del database (stereotipo «Database»).

4.2. Dipendenze e Comunicazioni

- bibliotecaSys.jar → biblioteca_db tramite dipendenza «JDBC» (driver mysql-connector-j 9.4.0).
- GestorePrenotazioni → ServizioNotifiche → sistema SMTP esterno tramite Jakarta Mail (protocollo SMTP).
- Lo schema del database è auto-generato da Hibernate tramite hbm2ddl (nessun artefatto DDL manuale nel deploy).

5. Test Plan

In questa sezione viene descritto il piano di testing previsto per verificare il corretto funzionamento del sistema. Il piano è organizzato per casi d'uso ed è circoscritto ai soli Use Case effettivamente implementati. Non è prevista alcuna mappatura dei requisiti né la verifica della loro soddisfazione: l'obiettivo è validare il comportamento del sistema rispetto agli scenari descritti nelle relative Use Case Description.

5.1. Approccio al Testing

Il piano di test adotta i seguenti criteri:

- Test funzionali: verifica del comportamento del sistema rispetto agli Use Case implementati.
- Test di scenario: copertura dei flussi principali e alternativi descritti nelle Use Case Description.
- Test di integrazione: verifica dell'interazione tra i livelli (Swing UI → Controller → Hibernate → MySQL).
- **Ambiente:** database MySQL con schema biblioteca_db popolato con dati di esempio tramite Hibernate (hbm2ddl=create-drop nei test).

5.2. Casi di Test per Caso d'Uso

LoginUtente

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”.
- Nel database è presente almeno uno Studente registrato e almeno un Bibliotecario.
- L'applicazione è compilata e avviata.
- La schermata FormLogin è visibile a schermo.
- Il contatore dei tentativi falliti è azzerato.

Testing Configurations

ID	Descrizione	Inputs	Azioni	Atteso
TC-L01	Login studente con credenziali valide	e-mail: test.studente@unina.it password: pass123	Click su Accedi	FormStudente si apre, FormLogin viene chiuso, la Sessione contiene lo Studente loggato.
TC-L02	Login bibliotecario con credenziali valide	e-mail: test.bib@unina.it password: pass456	Click su Accedi	FormBibliotecario si apre, FormLogin viene chiuso, la Sessione contiene il Bibliotecario loggato.
TC-L03	Password errata	e-mail: test.studente@unina.it password: sbagliata	Click su Accedi	Popup Credenziali errate, login non eseguito, contatore incrementato a 1.
TC-L04	E-mail non registrata	e-mail: nessuno@uninanas.it password: pass789	Click su Accedi	Popup Credenziali errate, login non eseguito.
TC-L05	Blocco account dopo 3 tentativi falliti	Password errata per 3 volte consecutive	Click su Accedi per 3 volte	Al 3° tentativo: popup Account bloccato, bottone Accedi

				disabilitato, ulteriori tentativi restituiscono null senza interrogare il database.
TC-L06	Campi vuoti	email: (vuoto) password: (vuoto)	Click su Accedi	Login fallito, messaggio di errore visualizzato.

Registrazione Utente

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”.
- Nel database è già presente uno Studente con email: “gia.registrato@unina.it” e matricola: 11111 (per testare i duplicati).
- L’applicazione è avviata e la schermata FormRegistrazione è accessibile tramite il bottone “Registrati” di FormLogin.
- Il ruolo di default selezionato nel ComboBox è “Studente”.

Testing Configurations

IDz	Descrizione	Inputs	Azioni	Atteso
TC-R01	Registrazione studente con dati validi	nome=Mario, cognome=Rossi email=nuovo@unina.it password=pass123 ruolo=STUDENTE, matricola=99999	Click su Registrati	Popup Registrazione studente completata, nuovo record inserito nella tabella Studente del DB.
TC-R02	Registrazione bibliotecario con dati validi	nome=Anna, cognome=Verdi email=bib.nuova@unina.it password=pass456 ruolo=BIBLIOTECARIO, codice=8888	Click su Registrati	Popup Registrazione bibliotecario completata, nuovo record inserito nella tabella Bibliotecario del DB.
TC-R03	E-mail già registrata (studente)	email=gia.registrato@unina.it matricola=22222 altri dati validi, ruolo=STUDENTE	Click su Registrati	Popup E-mail già associata ad un altro studente, nessun record inserito nel DB.
TC-R04	Matricola già registrata	email=nuova.email@unina.it matricola=11111 altri dati validi, ruolo=STUDENTE	Click su Registrati	Popup Matricola già registrata, nessun record inserito nel DB.
TC-R05	E-mail non valida (senza @)	e-mail=emailsenzachiocciola altri dati validi	Click su Registrati	Popup E-mail non valida, registrazione non eseguita.
TC-R06	Nome vuoto	nome=(vuoto) altri dati validi	Click su Registrati	Popup Nome obbligatorio, registrazione non eseguita.

CreaSala

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”.
- Il bibliotecario è autenticato nel sistema con una sessione valida.
- La schermata FormCreaSala è visibile a schermo.
- Non esiste già una sala con lo stesso nome nel database.

Testing Configurations

ID	Descrizione	Inputs	Azioni	Atteso
TC-CS01	Creazione sala senza aree (flusso principale)	nome=Sala A, descrizione=Sala silenziosa numPostazioni=10 oraApertura=08:00, oraChiusura=20:00 presenzaAree=false	Click su Salva sala	Sala creata nel database con 10 postazioni, popup "Sala creata correttamente", redirect a FormBibliotecario.
TC-CS02	Orario apertura >= orario chiusura	nome=Sala C, numPostazioni=5 oraApertura=20:00, oraChiusura=08:00	Click su Salva sala	Popup "Orario apertura non valido", nessuna sala salvata nel database, form rimane aperto.
TC-CS03	Campo nome vuoto	nome=(vuoto), descrizione=Test numPostazioni=5 oraApertura=08:00, oraChiusura=20:00	Click su Salva sala	Popup "Sala non creata correttamente", form rimane aperto, nessun dato salvato nel database.
TC-CS04	Numero postazioni non valido	nome=Sala D numPostazioni=0 oppure valore non numerico	Click su Salva sala	Popup di avviso, form rimane aperto, nessun dato salvato nel database.
TC-CS05	Annullamento con conferma (flusso alternativo)	Dati parzialmente inseriti	Click su Annulla, poi conferma nel popup	Sistema torna a FormBibliotecario senza salvare alcun dato, nessuna entry aggiunta nel database.
TC-CS06	Somma postazioni aree superiore al totale	nome=Sala E, numPostazioni=4, presenzaAree=true area1=(SILENZIOSA, 3), area2=(CONSULTAZIONE, 3)	Click su Salva sala	Popup di avviso, sala non salvata, form rimane aperto.

EffettuaPrenotazione

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”; schema popolato tramite Hibernate.
- Lo Studente è autenticato e ha avviato “Effettua Prenotazione”; tramite Include(ConsultaFasceDisponibili) gli sono mostrate le fasce disponibili per sala/giorno/area.
- Nel database è presente una SalaStudio con più Postazione e Aree configurate.
- Sono presenti prenotazioni preesistenti tali da rendere alcune postazioni occupate su determinate fasce, così da forzare sia il caso “singola postazione copre tutte le fasce” sia il caso “nessuna postazione copre tutte le fasce contigue”.

Testing Configurations

ID	Descrizione	Inputs	Azioni	Atteso
TC-EP01	Base — conferma (Scenario Base 5.x)	Selezione soddisfacibile con una singola postazione.	Studente seleziona fascia/e, conferma.	Studente seleziona fascia/e, conferma.
TC-EP02	Alternativo — postazione unica copre fasce contigue (SA1 5.1)	Più fasce contigue selezionate; esiste una postazione disponibile su tutte.	Studente seleziona più fasce contigue, conferma.	Studente seleziona più fasce contigue, conferma.
TC-EP03	Alternativo — postazioni multiple (SA1 4.1–4.2)	Più fasce contigue; nessuna singola postazione le copre tutte.	Studente seleziona fasce, sistema chiede conferma per N prenotazioni separate, studente conferma.	Studente seleziona fasce, sistema chiede conferma per N prenotazioni separate, studente conferma.
TC-EP04	Alternativo — rifiuto conferma (SA1 7.1)	Scenario a postazioni multiple: sistema ha trovato la soluzione e chiede conferma.	Studente non conferma.	Studente non conferma.
TC-EP01	Base — conferma (Scenario Base 5.x)	Selezione soddisfacibile con una singola postazione.	Studente seleziona fascia/e, conferma.	Studente seleziona fascia/e, conferma.
TC-EP02	Alternativo — postazione unica copre fasce contigue (SA1 5.1)	Più fasce contigue selezionate; esiste una postazione disponibile su tutte.	Studente seleziona più fasce contigue, conferma.	Studente seleziona più fasce contigue, conferma.

ConsultaFasceDisponibili

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”; schema popolato tramite Hibernate con dati d’esempio.
- Nel database è presente almeno una SalaStudio con un numero noto di Postazione e orari di apertura definiti.
- Sono presenti Prenotazione in stati differenti (attiva, scaduta, annullata) distribuite su più fasce orarie.
- Uno Studente è autenticato e accede alla schermata di consultazione fasce orarie.
- La verifica si concentra sul calcolo dinamico della disponibilità e sulla coerenza dello stato (disponibile / quasi pieno / esaurito) per ciascuna fascia.

Testing Configurations

ID	Descrizione	Inputs	Azioni	Atteso
TC-CFD01	Elenco fasce mostrato — sala interamente libera	Nessuna prenotazione attiva per la sala/giorno/area selezionati.	Studente seleziona sala, giorno e area; visualizza fasce.	Studente seleziona sala, giorno e area; visualizza fasce.
TC-CFD02	Elenco fasce mostrato — occupazione parziale sotto soglia	Alcune postazioni prenotate; residui sopra la soglia “quasi pieno”.	Studente seleziona sala, giorno e area; visualizza fasce.	Studente seleziona sala, giorno e area; visualizza fasce.

TC-CFD03	Elenco fasce mostrato — occupazione parziale oltre soglia	Residui sotto la soglia “quasi pieno” ma > 0.	Studente seleziona sala, giorno e area; visualizza fasce.	Studente seleziona sala, giorno e area; visualizza fasce.
TC-CFD04	Elenco fasce mostrato — fascia esaurita	Tutte le postazioni prenotate (attive) per una FasciaOraria.	Studente seleziona sala, giorno e area; visualizza fasce.	Studente seleziona sala, giorno e area; visualizza fasce.
TC-CFD05	Elenco fasce mostrato — prenotazioni non attive non conteggiate	Prenotazioni annullate/scadute presenti sulla fascia.	Studente seleziona sala, giorno e area; visualizza fasce.	Studente seleziona sala, giorno e area; visualizza fasce.
TC-CFD06	Nessuna fascia disponibile	Sala/giorno/area privi di FasciaOraria configurata.	Studente seleziona sala, giorno e area.	Studente seleziona sala, giorno e area.

EffettuaCheckIn

Testing Setup

- Il database MySQL è in esecuzione con lo schema “biblioteca_db”; schema popolato tramite Hibernate.
- Uno Studente è autenticato in sessione tramite Sessione.getInstance().
- Nel database sono presenti Prenotazione associate allo Studente con stato = ATTIVA e dataCheckIn = oggi, così da popolare correttamente la lista mostrata nella form.
- Nel database è presente almeno una Prenotazione con stato = ATTIVA e fascia oraria non ancora scaduta, per testare il check-in con successo.

Testing Configurations

ID	Descrizione	Inputs	Azioni	Atteso
TC-CHK01	Base — check-in confermato	Prenotazione selezionata con stato = ATTIVA e fascia oraria non scaduta.	Studente seleziona la prenotazione, clicca “Effettua Check-In” e conferma.	Studente seleziona la prenotazione, clicca “Effettua Check-In” e conferma.
TC-CHK02	Alternativo — nessuna selezione	Lista prenotazioni visibile; nessuna riga selezionata.	Studente clicca “Effettua Check-In” senza selezionare nulla.	Studente clicca “Effettua Check-In” senza selezionare nulla.
TC-CHK03	Alternativo — prenotazione scaduta	Studente seleziona una prenotazione con stato = SCADUTA.	Studente clicca “Effettua Check-In”.	Studente clicca “Effettua Check-In”.
TC-CHK04	Alternativo — rifiuto conferma	Prenotazione valida selezionata; dialog di conferma aperto.	Studente sceglie “No” nel dialog di conferma.	Studente sceglie “No” nel dialog di conferma.
TC-CHK05	Alternativo — annulla	Form aperta; nessuna azione di check-in avviata.	Studente clicca “Annulla”.	Studente clicca “Annulla”.
TC-CHK01	Base — check-in confermato	Prenotazione selezionata con stato = ATTIVA e fascia oraria non scaduta.	Studente seleziona la prenotazione, clicca “Effettua Check-In” e conferma.	Studente seleziona la prenotazione, clicca “Effettua Check-In” e conferma.

6. Manuale Utente e Istruzioni di Installazione

6.1. Repository del Progetto

Il codice sorgente del progetto è disponibile al seguente indirizzo:

https://github.com/LeonardoVernieri/Gruppo_IS_Locus.git

La struttura del repository è la seguente:

- Documentation/ — documentazione del progetto in formato .doc e .pdf.
- VisualParadigm/ — file .vpp del progetto Visual Paradigm.
- JavaProject/ — progetto Maven con il codice sorgente e i test.

6.2. Prerequisiti

- JDK 26 o superiore (compatibile con java.lang.classfile di Java 26).
- Maven 3.8+.
- MySQL 8.x — istanza locale con utente avente permessi di creazione schema.
- (Facoltativo per test email) accesso SMTP esterno oppure GreenMail in modalità standalone.

6.4. Build e Avvio

Dalla directory JavaProject/, eseguire i seguenti comandi:

```
mvn clean package -f JavaProject/IDS_Gr02/pom.xml  
java -jar JavaProject/IDS_Gr02/target/Locus-executable.jar
```

In alternativa, aprire il progetto in IntelliJ IDEA e avviare la classe Main direttamente dall'IDE.

6.5. Funzionalità Principali — Guida Rapida

6.5.1. Registrazione e Login

All'avvio, l'applicazione presenta la schermata di login. Gli utenti non ancora registrati possono accedere alla schermata di registrazione, selezionare il proprio ruolo (Studente o Bibliotecario) e inserire i dati richiesti. Una volta registrati, possono effettuare il login con email istituzionale e password.

6.5.2. Interfaccia Studente

- Effettua Prenotazione: selezionare sala, data e fascia oraria desiderata (o più fasce contigue) e confermare.
- Le mie Prenotazioni: visualizzare l'elenco delle prenotazioni con stato e dettagli; possibilità di annullare prenotazioni attive.
- Effettua Check-In: nella sezione prenotazioni, selezionare una prenotazione attiva e confermare il check-in (entro 4 ore dall'inizio).
- Profilo Personale: consultare lo storico completo delle prenotazioni e il numero totale di accessi alla biblioteca.

6.5.3. Interfaccia Bibliotecario

- **Gestione Sale:** creare nuove sale (con o senza suddivisione in aree), modificare e eliminare sale esistenti.
- **Prenotazioni Sala:** selezionare una sala e visualizzare l'elenco delle prenotazioni associate.
- **Monitoraggio Sale:** visualizzare in tempo reale postazioni occupate/libere, prenotazioni attive del giorno e statistiche di occupazione.
- **Storico Prenotazioni:** consultare lo storico filtrando per sala o per studente.

6.5.4. Notifiche

Il sistema invia e-mail automatiche agli studenti nei seguenti casi: conferma di una nuova prenotazione, annullamento di una prenotazione (con indicazione della causa), promemoria il giorno precedente alla prenotazione, scadenza della prenotazione per mancato check-in. Per il corretto funzionamento delle notifiche è necessario configurare i parametri SMTP nel file di configurazione dell'applicazione.